Department of Computer Science

BSc (Hons) Computer Science

Lattice: Privacy-Preserving Sealed-Bid Auctions over Veilid with MPC

Anker Rasmussen

anker.rasmussen@city.ac.uk

Project Consultant: Martin Brain

Client: Martin Brain (Academic Client)

Category: Academic Client Project

Subcategory: Application Development

Word count: 9,807 / 12,000

Proprietary Interests: None

Licence: MPL-2.0

11 April 2026

Abstract

This project addresses privacy in online sealed-bid auctions, where current systems require either a trusted auctioneer who sees all bids or a public ledger that exposes them. The central question is whether a decentralised auction can protect both bid values and participant network identities without any trusted intermediary. Prior work on MPC-based auctions has relied on small sets of known, trusted servers communicating over direct TCP connections, leaving the combination of multi-party computation with transport-layer anonymity unexplored. This project combined the MASCOT protocol (which computes the auction winner without revealing losing bids, even if all but one party is corrupt) with Veilid, a peer-to-peer framework whose onion-routed private routes hide participants' IP addresses from each other and from network observers. A working prototype was implemented in Rust ($\sim$15,000 lines), using Veilid's distributed hash table as the sole coordination substrate, and benchmarked across devnets of 40–80 nodes with both the MASCOT and Shamir protocols under party counts from 3 to 20. The key finding is that MPC over a privacy-preserving overlay is feasible: the tunnel sustained hundreds of megabytes of MASCOT traffic and produced correct results at up to 12 parties, with the 15–19$\times$ overhead from onion routing representing the quantifiable cost of hiding participants' network identities. This cost that is fundamental to the privacy model but acceptable for the small party counts typical of real auctions.

Acknowledgements

I would like to thank my project consultant, Martin Brain, for his outstanding support throughout this project. Beyond setting a genuinely interesting academic client project, he provided excellent ideas, discussed architectural implementations in depth, and steered me away from rabbit holes that would have consumed time and returned nothing. His guidance shaped the project at every stage.

I would also like to thank Christien Rioux (DilDog), lead developer of the Veilid framework, and the Veilid Discord community. The sheer volume of information available on the Discord, where other developers had run into exactly the same issues and DilDog responded with recommended architectural choices, accelerated development enormously and directly informed the design of this system.

List of Figures

4.1 Playground devnet topology (subset shown). Node 0 (double border) is the bootstrap server; after initial discovery all nodes form a P2P mesh. Lattice market nodes (shaded) join with offsets above the devnet range. All nodes run on a single machine with `LD_PRELOAD` IP spoofing. 10

4.2 Auction phase flowchart. Rectangles are processing phases; diamonds are decision points; rounded boxes are terminal states. The "no" paths from route collection and readiness barrier represent retry loops (DHT re-poll and route refresh respectively). MPC failure leads to the Failed terminal. Each participant independently tracks this flow for every auction. 12

4.3 DHT record hierarchy. Shaded boxes are directory records; dashed boxes show subkey layout. Bidders (B_1–B_n, dashed arrows) each merge their announcement into the listing's G-Set CRDT (subkey 1) and own an individual bid record. Each node also maintains a local copy of the G-Set in memory; periodic DHT reads merge remote state into the local copy, ensuring convergence. Each bid record's subkey 1 carries the MPC route blob for DHT-backed route exchange. 13

5.1 The MPC auction program (`auction_n.mpc`), written in the MP-SPDZ DSL. The comparison on line 16 and conditional select on lines 17–18 are computed over secret-shared values - $sint$, the provided primitive for an (appropriately named) secret int within SPDZ. No party sees intermediate results. The `reveal_to(0)` calls on lines 21–22 ensure that only party 0 (the seller) learns the winner identity and winning bid. 18

5.2 Direct MPC computation time (localhost TCP, no Veilid overlay). MASCOT scales quadratically with party count due to pairwise OT setup; Shamir scales linearly. Error bars show ± 1 standard deviation over 10 iterations. 21

5.3 Total auction time over Veilid using MASCOT (median of successful runs, $\pm 1\sigma$). Route exchange and MPC tunnel latency dominate. Larger devnets reduce success rates (see Figure 5.8) but successful runs show similar wall-clock times. 22

5.4 Total auction time over Veilid using Shamir secret sharing (median of successful runs, $\pm 1\sigma$). The 40-node devnet has insufficient relay capacity to maintain routes for all 20 parties ($N{=}20$: 0/10 success). Only successful iterations are included; timeouts are excluded. 22

5.5 Time breakdown for Shamir auctions on a 40-node devnet. Route exchange through Veilid private routes dominates total time; the MPC computation itself (tunnelled over Veilid `app_call`) accounts for less than 10% at most party counts. 23

5.6 MPC computation time: direct TCP vs. tunnelled through Veilid private routes (40-node devnet, median of successful runs). The Veilid tunnel adds approximately $15{-}19\times$ overhead for MASCOT and $10{-}30\times$ for Shamir, attributable to per-round `app_call` latency through onion-routed hops. 23

5.7 Per-party data sent during MPC computation (direct TCP). MASCOT's pairwise oblivious transfer requires $O(N^2)$ bandwidth growth (37 MB at $N{=}3$ to 653 MB at $N{=}10$), while Shamir's secret-sharing approach grows linearly (0.02 MB at $N{=}3$ to 1.4 MB at $N{=}20$). MASCOT benchmarks beyond $N{=}12$ were not feasible: the host desktop environment ran out of memory under the combined load of devnet nodes and MPC processes. . . 24

5.8 Success rate by party count and devnet size. MASCOT maintained 100% success on the 40-node devnet up to $N{=}12$ but degraded on larger devnets. Shamir scaled further in party count but dropped to 0% at $N{=}20$ on 40 nodes. 24
5.9 MPC computation time vs. total non-route time (40-node devnet). Points above the $y{=}x$ line indicate overhead from Veilid tunnel latency. MASCOT incurs larger absolute overhead due to its higher round count. 25

List of Tables

1.1 Comparison of auction privacy models (eBay Help 2025; OpenSea 2025a; U.S. General Services Administration 2025). 1
1.2 Security policy: what is protected and from whom. 2
1.3 Adversary model. 2
2.1 Project deliverables. 4
3.1 Overlay network comparison. 7
5.1 Data volumes tunnelled through Veilid per auction. Each `app_call` carries up to 32 KB of payload. MASCOT transfers roughly 1,000$\times$ more data than Shamir at $N{=}3$, growing to 1,500$\times$ at $N{=}10$ due to $O(N^2)$ pairwise OT. 19
5.2 Test matrix. 20
5.3 Per-round MPC latency: direct TCP vs. Veilid tunnel (40-node devnet). Per-round latency is computed as MPC wall-clock time divided by round count. Shamir $N{=}20$ is absent because no trial succeeded on the 40-node devnet. 25
5.4 MASCOT $N{=}3$: Veilid tunnel vs. direct TCP (40-node devnet, 10 successful trials). The tunnel adds 17.1 ms/round of overhead, dominated by Veilid relay latency. 28
5.5 Summary of residual information leakage channels. 30

Glossary

AES-256-GCM	Advanced Encryption Standard with 256-bit key in Galois/Counter Mode. An authenticated encryption scheme providing both confidentiality and integrity.
Commitment scheme	A two-phase cryptographic protocol: a party first publishes a binding commitment (e.g. a hash), then later reveals the committed value. Used here for sealed bids.
CRDT	Conflict-Free Replicated Data Type. A data structure designed for distributed systems where multiple nodes can update their own copy independently, and all copies are guaranteed to converge to the same state without coordination. Used in this project as a G-Set (grow-only set) for the bid announcement registry, ensuring that concurrent bid announcements from different nodes merge correctly without conflicts.
DHT	Distributed Hash Table. A decentralised key–value store where records are spread across participating nodes rather than held on a single server. Lookups are routed through the network so that no single node needs to store everything.
E2E	End-to-end. In this report, refers to tests exercising the full system stack including real Veilid networking.
NAT	Network Address Translation. Relevant because Veilid must traverse NATs (carrier-grade or regular) to establish peer connections.
P2P	Peer-to-Peer. A network model with no central server; all participants are equal.
SHA-256	Secure Hash Algorithm producing a 256-bit digest. Used for bid commitments: $C = \text{SHA-256}(\textit{bid} \| \textit{nonce})$.
TCP	Transmission Control Protocol. MP-SPDZ parties communicate over TCP; the MPC tunnel proxies TCP over Veilid.
Dishonest majority	An MPC security model tolerating corruption of up to $N-1$ of N parties. Stronger than honest majority but more expensive. Used by MASCOT.
HE	Homomorphic Encryption. A form of encryption that allows mathematical operations to be performed on encrypted data without decrypting it first. The result, when decrypted, matches the outcome of performing the same operations on the plaintext. Used in some MPC protocols (e.g. the original SPDZ) as an alternative to oblivious transfer.
Honest majority	An MPC security model requiring that fewer than $N/2$ parties are corrupt. Used by Shamir secret sharing.

Integer triples	Pre-computed tuples (a, b, c) where $c = a \cdot b$, shared among MPC parties. Also called Beaver triples. Used in SPDZ/MASCOT to reduce online multiplication to additions and openings, moving expensive computation to an offline preprocessing phase.
MAC	Message Authentication Code. In SPDZ/MASCOT, every secret-shared value carries a MAC tag under a global key (α) that is itself secret-shared among all parties. When a value is opened, all parties verify the MAC; any tampering by a corrupt party causes the check to fail and the protocol to abort. This is what gives MASCOT its malicious security.
Malicious security	An MPC security model where corrupt parties may deviate arbitrarily from the protocol. Stronger than semi-honest security.
MASCOT	Malicious And Secure Computation using Oblivious Transfer. A dishonest-majority MPC protocol with malicious security (Keller, Orsini and Scholl 2016).
MEV	Miner-Extractable Value. Profit that miners or validators can extract by reordering, inserting, or censoring transactions within a block before it is finalised. In the context of on-chain auctions, MEV enables front-running (seeing a pending bid and placing one's own bid ahead of it) and sandwich attacks (placing transactions both before and after a victim's transaction to artificially move the price and profit from the difference) (Daian et al. 2019).
MPC / SMPC	(Secure) Multi-Party Computation. Protocols enabling joint computation over private inputs without revealing them to other parties.
MP-SPDZ	Multi-Protocol SPDZ. A framework providing multiple MPC protocol backends from a single high-level program (Keller 2020).
OT	Oblivious Transfer. A cryptographic primitive where a sender holds N messages and a receiver selects one to receive. The sender learns nothing about which message was chosen, and the receiver learns nothing about the other messages. MASCOT uses OT extensively to generate the correlated randomness (integer triples) needed for secure multiplication.
Replicated secret sharing	The simplest multi-party MPC protocol, supporting exactly three parties. Each party holds two of the three shares of every secret value, so any two parties can reconstruct it. Requires an honest majority (at most one corrupt party). Used during early development as a "hello world" to validate the MPC integration pipeline.
Semi-honest security	An MPC security model assuming corrupt parties follow the protocol but try to learn extra information from intermediate values. Weaker than malicious security.

SPDZ	"Speedz." A family of MPC protocols with dishonest-majority security, originally based on somewhat homomorphic encryption (Damgård, Pastro, Smart and Zakarias 2011).
SSS	Shamir's Secret Sharing. A threshold scheme distributing a secret among N parties such that any $t+1$ shares reconstruct it, but t or fewer reveal nothing (Shamir 1979).
TEE	Trusted Execution Environment. A hardware-isolated processor region for secure computation. An alternative to MPC with different trust assumptions (Li et al. 2023).
Threshold cryptography	A cryptographic technique where a secret key is split among N parties such that at least $t+1$ must cooperate to perform an operation (e.g. decryption or signing). No single party can act alone, removing single points of trust. Proposed as future work to eliminate the seller's exclusive control over decryption key release.
`app_call`	Veilid API for request–response messaging with confirmed delivery. Used for all reliable point-to-point communication in this system.
`app_message`	Veilid API for fire-and-forget messaging without delivery confirmation. Used only for non-critical broadcasts.
I2P	Invisible Internet Project. An anonymity network similar to Tor but designed for internal ("darknet") services rather than accessing the public internet. Uses garlic routing, a variant of onion routing that bundles multiple messages together for efficiency.
IPFS	InterPlanetary File System. A content-addressed P2P storage network with no built-in privacy layer.
Onion routing	A network anonymity technique where a message is encrypted in multiple layers (like the layers of an onion), one for each relay node on the path. Each relay strips one layer of encryption and forwards the result to the next hop. No single relay knows both the sender and the final destination, providing sender anonymity.
Overlay network	A virtual network built on top of physical infrastructure, where peers form logical links independent of their IP topology.
Private route	A Veilid construct: an onion-routed path through relay nodes, represented as an opaque blob that can be published and imported by remote peers.
Relay node	A peer in the Veilid overlay that forwards messages on behalf of other nodes without being the sender or final recipient. When a private route is constructed, it passes through one or more relay nodes; each relay strips a layer of encryption and forwards the result. The more relay nodes (hops) in a route, the harder it is to trace the message back to its origin, but each additional hop adds latency.

Hop	A single step in an onion-routed path. A message travelling through a private route with two hops passes through two relay nodes between sender and receiver. Veilid defaults to two hops per private route.
Route-in-DHT	A coordination pattern where a node writes its private route blob to a DHT record so that remote peers can discover and import it without direct messaging. Adapted from Stigmerge (Marshall 2024).
Stigmergy	Indirect coordination through a shared environment rather than direct communication. The route-in-DHT pattern is an instance of stigmergy.
Tor	The Onion Router. An anonymity network using onion routing and exit nodes, designed for client–server communication rather than P2P.
Tunnelling	Encapsulating one protocol inside another to transport it across an intermediate network. Here: TCP-over-Veilid for MPC traffic.
Veilid	A privacy-first, open-source P2P application framework providing onion-routed messaging, a secure DHT, and pseudonymous public-key identities (no real names or IP addresses are exposed to other participants) (Veilid Project 2025c).
English auction	An ascending-bid auction where the current highest bid is visible and bidders compete openly until no higher bid is placed.
First-price sealed-bid	An auction where bids are secret; the highest bidder wins and pays their bid price.
Front-running	An attack where an adversary observes a pending transaction (e.g. a bid) before it is finalised and submits their own transaction ahead of it to profit. In blockchain contexts, miners or validators can reorder transactions within a block to achieve this. See also MEV.
Sealed-bid auction	An auction where bids remain confidential during submission. Contrasts with open/English auctions.
Vickrey auction	A second-price sealed-bid auction: the highest bidder wins but pays the second-highest bid.
Bincode	A compact binary serialisation format. Used for ephemeral wire messages (`AuctionMessage` enum).
CBOR	Concise Binary Object Representation. A self-describing binary format used for DHT-stored records (listings, bids).
Dioxus	A React-inspired Rust framework for building cross-platform desktop applications (v0.7.2 used in this project).
Lattice	The peer-to-peer sealed-bid auction marketplace built in this project. Implemented in Rust, runs on the Veilid network, and uses MP-SPDZ for MPC-based winner determination.

LD_PRELOAD A Unix mechanism for intercepting library calls at runtime. Used here with `libipspoof.so` to give each Veilid node a unique public-facing IP address on a single machine. Veilid requires each node to appear as a distinct host on the public internet; without IP spoofing, multiple nodes on the same device fail to form separate routing identities.

MPL-2.0 Mozilla Public License 2.0. A file-level copyleft licence: modifications to MPL-licensed files must be published under MPL. Used by Veilid.

Contents

Abstract

Acknowledgements

1 Introduction **1**
1.1 Problem Statement . . . 1
1.2 Security Policy . . . 1
1.3 Project Objectives . . . 3
1.4 Beneficiaries . . . 3
1.5 Report Structure . . . 3

2 Output Summary **4**

3 Context **5**
3.1 Secure Multi-Party Computation . . . 5
3.1.1 Shamir Secret Sharing (Honest Majority) . . . 5
3.1.2 MASCOT (Dishonest Majority) . . . 5
3.2 Veilid: Privacy-Preserving Peer-to-Peer Networking . . . 6
3.2.1 What Veilid Protects (and What It Does Not) . . . 6
3.2.2 Bootstrap and Network Formation . . . 8
3.3 Stigmerge and the Route-in-DHT Pattern . . . 8
3.4 Auction Privacy in Context . . . 8

4 Method **10**
4.1 Development Approach . . . 10
4.1.1 Infrastructure . . . 10
4.2 Architecture . . . 11
4.2.1 Auction State Machine . . . 11
4.2.2 DHT Record Hierarchy . . . 11
4.2.3 MPC Tunnel . . . 14
4.2.4 Winner Verification and Content Delivery . . . 14
4.3 Technology Choices . . . 14
4.3.1 Why Veilid . . . 14
4.3.2 Why MP-SPDZ . . . 14
4.3.3 Why MASCOT and Shamir . . . 14
4.3.4 Serialisation Strategy . . . 15
4.3.5 UI Framework . . . 15
4.4 Testing Strategy . . . 15
4.5 Reuse and Attribution . . . 16

5 Results **17**
5.1 System Architecture . . . 17
5.1.1 DHT as Shared State . . . 17
5.2 Auction Protocol . . . 17

5.3 MPC Integration . . . 18
5.3.1 The Auction Program . . . 18
5.3.2 Tunnel Architecture . . . 19
5.3.3 Route Lifecycle Challenges . . . 19
5.4 User Interface . . . 20
5.5 Testing and Verification . . . 20
5.5.1 Test Coverage . . . 20
5.5.2 Integration Test Results . . . 20
5.5.3 End-to-End Test Results . . . 20
5.6 Performance Benchmarks . . . 21
5.6.1 Direct MPC Baseline . . . 21
5.6.2 Veilid Tunnel Overhead . . . 21
5.6.3 Total Auction Time . . . 26
5.6.4 Reliability and Success Rates . . . 26
5.6.5 Devnet Size Effects . . . 27
5.6.6 Hardware and Software Limits . . . 27
5.7 Security Analysis . . . 28
5.7.1 Threat Model . . . 28
5.7.2 Timing Side Channel on Party Assignment . . . 28
5.7.3 Seller as Single Point of Trust . . . 28
5.7.4 Winner Identity Linkage . . . 29
5.7.5 DHT Metadata Leakage . . . 29
5.7.6 MPC Route Correlation . . . 29
5.7.7 Small-N Security Considerations . . . 29
5.7.8 Summary of Side Channels . . . 29

6 Conclusions and Discussion **31**
6.1 Objectives Revisited . . . 31
6.2 Discussion . . . 31
6.3 Legal, Social, Ethical and Professional Issues . . . 32
6.4 Future Work . . . 32
6.5 Reflections . . . 33

A Project Definition Document **37**

B Reuse Summary **54**

C Supervisory Meeting Minutes **55**

D Generative AI Usage **58**

E Veilid Developer Correspondence **63**

1 Introduction

1.1 Problem Statement

Since the inception of the digital age, many established industries have moved online. Auction systems in particular have seen a massive surge in popularity due to the low barrier to entry for both buyers and sellers: eBay sold $7.2 million worth of goods within a year of launch (eBay Inc. 2025b) and today hosts 2.5 billion live listings across 135 million active buyers worldwide (eBay Inc. 2025a), and this is only one platform. However, their security models have not kept pace. The move from physical sealed envelopes to digital platforms introduced new attack surfaces, including bid visibility, platform trust, and metadata exposure. Existing systems address these concerns incompletely, if at all.

Popular online marketplaces follow three distinct patterns. eBay uses open proxy bidding: the current highest bid is visible throughout the auction and the platform operator sees all bids (eBay Help 2025). Facebook Marketplace is a listing-plus-messaging workflow with no formal auction protocol and weak buyer protections (Meta/Facebook Help Center 2025). Blockchain marketplaces such as OpenSea (OpenSea 2025a) use English style auctions where every bid is an on-chain transaction, publicly visible on all supported chains (OpenSea 2025b) and permanently recorded solving the trusted-auctioneer problem via smart contracts, but providing zero bid privacy.

Government procurement offers a contrasting model. Under the US Federal Acquisition Regulation, sealed bidding keeps all bids confidential until a public opening event (U.S. General Services Administration 2025). Bids are private *during* submission, but a trusted auctioneer (the contracting officer) sees every bid at opening and determines the winner. Sealed bidding thus protects bids from *other bidders* but not from the *auctioneer*.

Daian et al. demonstrated that even blockchain-based commit-reveal schemes are vulnerable to front-running and miner-extractable value (MEV) (Daian et al. 2019), where transaction ordering can be exploited during the reveal phase. Cryptographic sealed-bid auctions can hide non-winning bids entirely (Sako 2000), and multi-party computation (MPC) can remove the need for a trusted auctioneer altogether (Bogetoft et al. 2009). However, deploying MPC over a transport layer that also hides participants' network identities remains largely unexplored.

System	Bids visible?	Trusted auctioneer?	Decentralised?	Bidder identity
eBay	Yes (open ascending)	Yes	No	Real name + address
OpenSea	Yes (on-chain)	No (contract)	Yes	Pseudonymous (wallet, IP visible)
US govt. sealed bid	No (until opening)	Yes (sees all)	No	Real name (registered)
Lattice(This project)	**No (MPC)**	**No**	**Yes**	**Pseudonymous (key, IP hidden)**

Table 1.1: Comparison of auction privacy models (eBay Help 2025; OpenSea 2025a; U.S. General Services Administration 2025).

1.2 Security Policy

Privacy in this system meant protecting specific information from specific adversaries. Table 1.2 summarises the security policy; Chapter 3 expands on the threat model in the context of Veilid's network-layer protections.

What is protected	From whom	Mechanism
Non-winning bid values	All parties (incl. seller)	MPC: only the maximum is revealed to party 0
Bidder network identity (IP)	Network observers, other bidders	Veilid private routes (onion-routed, multi-hop)
Listing content before purchase	Non-winners	AES-256-GCM; key released only to verified winner
Bid existence and timing	Partially protected	DHT participation metadata remains visible

Table 1.2: Security policy: what is protected and from whom.

The *adversary model* considered four types, summarised in Table 1.3.

Adversary	Capability	Goal
Honest-but-curious bidder	Follows protocol, observes all available data	Learn other bids or bidder identities
Malicious seller	Controls listing, party 0 output, key release	Grief (withhold key), collude with preferred bidder
Passive network observer	Monitors DHT writes and message patterns	Deanonymise participants
Colluding parties	Multiple participants share private state	Break MPC threshold, reconstruct secret inputs
Dishonest participant	Participates in auction, observes protocol	Deanonymise other bidders or link pseudonyms to IPs

Table 1.3: Adversary model.

Key *trust and design assumptions*:

- The seller is semi-trusted: as MPC party 0 they learn the winning bid value and receive an ephemeral onion route to the winner, but not the winner's network identity. The seller controls decryption key release and can grief by withholding the key.
- Veilid's private routing provides transport-layer anonymity; the effective anonymity set depends on both the number of relay nodes available in the overlay and the number of hops configured per private route. In this project's devnet; due to the devnet size, the hops default to 1. More relay nodes increase the pool of possible intermediaries; more hops make traffic analysis harder but add latency per round trip.
- Liveness requires all MPC parties to remain online throughout computation. A single party going offline aborts the auction.
- The MPC security guarantee depends on the protocol backend: MASCOT (Keller, Orsini and Scholl 2016) tolerates up to $N-1$ corrupt parties (dishonest majority)- and this is justified by the seller getting extra information in comparison to the bidding participants. Shamir (Shamir 1979) requires an honest majority of at least $\lfloor N/2 \rfloor + 1$ honest parties.
- The system operates on a private devnet, not the public Veilid network. All nodes are controlled by the tester; no real-world adversaries or network conditions are present or have been tested.
- Each Veilid node is assumed to have a unique public IP address. The devnet simulates this via LD_PRELOAD IP spoofing (Section 3.2.2).
- Veilid supports only one active private route per node at a time. MPC sessions are therefore sequential; concurrent auctions on the same node are not possible.

- Party assignment is deterministic (sorted by bid timestamp). Any deterministic assignment from public data (timestamps, public keys, or otherwise) is publicly derivable and constitutes a known side channel. With MASCOT this is low severity, as the protocol is secure regardless of party identity; with Shamir it is more concerning if an attacker can target specific parties for corruption (Section 5.7).
- No Sybil resistance: Veilid itself provides no defence against a single adversary creating multiple pseudonymous identities. By extension, nothing prevents one party from placing multiple bids in the same auction under different keys. Mitigating this would require an identity layer that is out of scope.
- The auction is first-price only: the highest bidder wins and pays their bid.
- No payment or settlement layer is integrated. The auction ends at decryption key delivery; actual exchange of goods or funds is out of scope.

1.3 Project Objectives

The main objective was to deliver a working peer-to-peer marketplace on the Veilid network supporting sealed-bid listings with private content unlock for the winning bidder. Four sub-objectives were defined in the PDD (Appendix A):

1. **Listings and purchases.** End-to-end auction flow: create listing $\rightarrow$ submit bid $\rightarrow$ determine winner $\rightarrow$ complete purchase.

2. **Sealed-bid via MPC.** Integrate a multi-party computation protocol to select the highest valid bid without revealing non-winning bids.

3. **MPC-gated decryption.** Bind winner selection to content access: only the verified winner obtains the decryption key.

4. **Settlement on Monero testnet.** Simulate wallet initialisation, deposits, and release on a private Monero testnet. This objective was descoped as it was not inline with the research goals of the project, being more suited to a production system (Section 6.1).

1.4 Beneficiaries

The project consultant gained a proof-of-concept demonstrating Veilid's capabilities for a non-trivial use case. The Veilid development community benefited from a real application exercising the framework's private routing and DHT APIs. The project also contributed empirical data on the overhead of tunnelling MPC through a privacy-preserving overlay. Also contributed was the playground infrastructure to the Veilid community; as a seperate testing environment for users to test their own applications in a "sandbox".

1.5 Report Structure

Chapter 2 summarises the deliverables. Chapter 3 provides technical background on the building blocks: Veilid, MP-SPDZ, and the stigmerge coordination pattern, and expands the security policy into a full threat model. Chapter 4 describes the architecture, protocol design, and testing strategy. Chapter 5 presents verification: functionality, performance benchmarks comparing MASCOT and Shamir over Veilid, and a security analysis of residual information leakage. Chapter 6 gives a critical overview, reflections, and future work.

2 Output Summary

Table 2.1 summarises the project deliverables.

Output	Type / Size	Recipient	Report reference
Lattice application	Rust crate, ∼15k lines	Consultant, Veilid community	Sections 5.1–5.5
`auction_n.mpc`	MP-SPDZ DSL, 46 lines	Consultant	Section 5.3
Benchmark harness	Rust binary + 354 trials (CSV)	Consultant	Section 5.6
Devnet tooling	Makefile, playground orchestrator	Consultant, Veilid community	Section 4.1
Demo video	15-min narrated recording	Examiners	*Submitted separately*
This report	LaTeX source in repo	Examiners	—

Table 2.1: Project deliverables.

The Lattice application was a desktop program built with Dioxus 0.7.2 that embedded a Veilid node, provided a graphical interface for creating listings and placing bids, and orchestrated the full MPC-based winner-determination flow. A headless binary (`market-headless`) was created to reduce the friction of automated end-to-end testing without the overhead of running a GUI. The MPC protocol was drop-in: the auction program compiled against any MP-SPDZ backend, and the runtime protocol was selected via an environment variable. MASCOT and Shamir were benchmarked, but any MP-SPDZ protocol supporting N-party computation could be substituted without code changes.

The benchmark harness (`bench-auction`) automated multi-party auction trials across configurable party counts (3–20) and devnet sizes (40, 60, 80 nodes), collecting wall-clock time, MPC computation time, route-exchange latency, tunnel bandwidth, and success rates.

Reuse Summary

The application depended on Veilid (MPL-2.0) for networking, MP-SPDZ (BSD-3-Clause) for MPC execution, and Dioxus (MIT/Apache-2.0) for the desktop UI. No code was directly copied from reference applications; the `app_call` delivery pattern over private routes was adapted from Stigmerge (Marshall 2024), a Veilid-based file-sharing application. A detailed component-level reuse table is provided in Appendix B.

All source code, build scripts, benchmark data, and this report are available in the project repository at https://github.com/anker-rasmussen/Dissertation (including submodules). The Rust crate was not published on crates.io or any public registry; it is available only as source code in the repository.

3 Context

3.1 Secure Multi-Party Computation

MPC enables a set of parties to jointly compute a function over their private inputs without revealing those inputs to one another (Evans, Kolesnikov and Rosulek 2018). The following definitions use the formalisations in Evans et al. (2018) and Zhao et al. (2019). Two security models are central to this project.

3.1.1 Shamir Secret Sharing (Honest Majority)

Shamir's Secret Sharing (Shamir 1979) hides a secret s inside a random polynomial of degree t whose constant term is s. Each party receives the polynomial evaluated at a distinct point. Any $t+1$ parties can reconstruct s via interpolation, but t or fewer learn nothing about it.

When used for MPC, the threshold is set to $t < N/2$ (honest majority). Addition of shared values is free (each party adds its shares locally), while multiplication requires one round of communication per gate, giving $O(N)$ messages per multiplication. This makes Shamir efficient at large party counts. However, if the honest-majority assumption is violated ($t \geq N/2$ parties are corrupt), the corrupt parties can reconstruct every shared secret, and all privacy guarantees are lost.

3.1.2 MASCOT (Dishonest Majority)

The MASCOT protocol (Keller, Orsini and Scholl 2016) is built on the SPDZ framework (Damgård, Pastro, Smart and Zakarias 2011) and provides malicious security tolerating up to $N-1$ corrupt parties. The MPC program is expressed as an arithmetic circuit: each operation (addition, multiplication, comparison) is a gate, and the parties jointly evaluate the circuit by exchanging messages at each gate. MASCOT operates in two phases.

Offline phase (preprocessing). Before any inputs are known, the parties generate a pool of authenticated Beaver triples using oblivious transfer (OT). For each pair of parties (i, j), a series of OT instances produces correlated random values that, when combined across all pairs, yield a globally shared triple (a, b, c) with $c = a \cdot b$.

Every shared value is authenticated with a MAC under a global key α that is itself secret-shared among all parties (Damgård, Pastro, Smart and Zakarias 2011). No single party knows α, so no party can forge a valid share.

The pairwise nature of OT means the offline phase requires $O(N^2)$ communication: every pair of parties must exchange OT messages. This is the dominant cost and the reason MASCOT is significantly more expensive than Shamir, particularly at higher party counts.

Online phase (computation). Addition of shared values is free (each party adds locally). Multiplication consumes one pre-computed Beaver triple per gate: the parties open masked values that reveal nothing about the inputs, then combine the result with the triple to obtain a sharing of the product.

The key difference from Shamir is what happens when a value is *opened* (revealed to all parties). Before accepting an opened value v, all parties perform a MAC check: each party i computes $m_i - v \cdot \alpha_i$ and the parties jointly verify that these sum to zero. If any corrupt party modified their share, the MAC check fails with overwhelming probability and the protocol aborts. This is what provides malicious security: corrupt parties can deviate arbitrarily from the protocol, but any cheating is detected before it affects the output.

A third protocol, replicated secret sharing, splits each secret into three additive shares and gives each of the three parties two of them. It is the simplest multi-party scheme but is restricted to exactly three parties with an honest majority. It was used during early development to validate the MPC integration pipeline before moving to MASCOT and Shamir for evaluation.

The MP-SPDZ framework (Keller 2020) provides academic production-grade implementations of all three protocols, allowing the same high-level auction program to be compiled and executed under different protocol backends. This was central to the evaluation in Chapter 5, where the same program was benchmarked under MASCOT and Shamir with identical inputs. Other MP-SPDZ backends (semi-honest two-party, BMR garbled circuits, etc.) were not tested because they either do not support $N > 2$ parties or solve a different threat model than the one defined in Section 1.2.

The Danish sugar-beet auction (Bogetoft et al. 2009) remains the most prominent real-world MPC deployment for auctions, using a three-party honest-majority protocol. That system relied on a small number of known, trusted intermediaries and a direct TCP network. This project extended the concept to an untrusted, peer-to-peer setting where the honest-majority assumption may not hold, motivating the comparison between MASCOT and Shamir.

Zhao et al. (Zhao et al. 2019) survey MPC theory and applications. Feng and Yang (Feng, Dengguo and Yang, Kang 2022) provide concrete efficiency benchmarks, noting that the choice between honest and dishonest majority remains the dominant factor in practical performance. Li et al. (Li et al. 2023) compare MPC with trusted execution environments (TEEs), concluding that MPC provides stronger trust assumptions at the cost of higher communication overhead.

3.2 Veilid: Privacy-Preserving Peer-to-Peer Networking

Veilid is described by its developers as

> "an open-source, peer-to-peer application framework designed to build privacy for all" (Veilid Project 2025b).

Each application embeds a Veilid node into a global overlay where peers are equal (no privileged relays), connections are end-to-end encrypted, and private routing obscures network locations.

Three capabilities made Veilid suitable for this project:

1. **Addressable encrypted endpoints** keyed by public keys rather than IP addresses, eliminating direct IP exposure.
2. **A secure DHT** for publishing listings, bids, and discovering auction participants.
3. **Onion-routed private routes** for point-to-point messaging that limit metadata exposure during bid submission and MPC tunnel traffic.

3.2.1 What Veilid Protects (and What It Does Not)

Veilid's private routes provide network-layer anonymity: a sender constructs a multi-hop route through relay nodes, and the receiver sees only the last hop's identity. This protects participants' IP addresses from each other and from passive network observers. However, several limitations were relevant to the security policy:

- **Anonymity set**: Privacy depends on the number of available relay nodes. On a small devnet (20–80 nodes), the anonymity set is correspondingly small. On the public Veilid network this would be larger but remains unquantified.

- **DHT metadata**: Writing a bid record to the DHT is observable by DHT nodes. The *number* and *timing* of bids are visible even though *values* are encrypted.
- **Route liveness**: Private routes are application-managed and can die silently. There is no notification when a remote route becomes invalid; sending to a dead route fails only after a timeout. This had significant implications for MPC integration (Section 5.3).
- **IP leakage without private routes**: If a node communicates using unsafe routing (direct node-to-node), its IP address is exposed on the wire. All benchmarked messaging in this system used safe routing exclusively.

Table 3.1 compares Veilid with three alternative overlay networks that offer similar privacy properties.

Property	**Tor**	**I2P**	**IPFS**	**Veilid**
Primary model	Client–server (exit nodes)	Internal services	Content-addressed storage	P2P application framework
Anonymity routing	Onion circuits (3 hops)	Garlic routing	None (cleartext DHT)	Onion-routed private routes
Built-in DHT	Hidden-service descriptors	LeaseSet/NetDB	Content routing	General-purpose key–value
Embeddable library	No (SOCKS proxy)	Partial (I2CP)	Yes (libp2p)	Yes (veilid-core)
P2P messaging	Via hidden services	Via streaming lib	No	`app_call` / `app_message`

Table 3.1: Overlay network comparison.

Tor (Dingledine, Mathewson and Syverson 2004) provides strong anonymity through onion-routed circuits but is designed for the client–server model: clients reach clearnet servers via exit nodes, or hidden services via rendezvous points. Building a peer-to-peer auction on Tor would require running each participant as a hidden service, managing circuit lifecycle externally, and implementing DHT storage separately. Tor's hidden services also impose a minimum of six hops per connection (three per side), adding latency that compounds across MPC rounds.

I2P (Zantout and Haraty 2011) is closer to a P2P model, using garlic routing to bundle messages for internal ("eepsite") services. Its streaming library supports bidirectional TCP-like connections between I2P destinations. However, I2P lacks a general-purpose DHT for application data (its NetDB stores only router and destination metadata), and its ecosystem is smaller with fewer maintained client libraries.

IPFS (Benet 2014) provides content-addressed storage over a Kademlia-based DHT and is well suited for data distribution. However, it provides no anonymity: DHT operations and data transfers expose participants' IP addresses. Adding a privacy layer would require tunnelling IPFS through Tor or I2P, combining the limitations of both.

Veilid was selected because it provides all three capabilities required by the application in a single embeddable library: a general-purpose DHT for storing listings and bids, onion-routed private routes for anonymous messaging, and a peer-to-peer architecture where all nodes are equal with no privileged relays or exit nodes. This avoided the need to compose multiple frameworks (e.g. IPFS for storage plus Tor for anonymity) and the integration complexity that would entail.

3.2.2 Bootstrap and Network Formation

A Veilid network requires at least one *bootstrap node*: a well-known server whose address is configured into every joining node so that they can discover initial peers and populate their routing tables (Veilid Project 2025a). On the public Veilid network, bootstrap nodes are discovered via DNS TXT records. For development and testing, a private devnet must provide its own bootstrap infrastructure.

Crucially, Veilid expects each node to present a unique public IP address. Its routing-domain logic classifies incoming connections by IP range and drops traffic from addresses it considers local or duplicate (Veilid Project 2025c). Veilid also implements its own STUN for nodes behind some level of NAT, but nodes behind NAT are not deemed "optimal", and recieve network connection penalties accordingly (cannot act as bootstrap/less priority for relay). This means that running multiple nodes on a single machine as required for a local devnet fails unless each node appears to be on the public internet and not behind a NAT layer. A community-contributed development network (Jackson 2024) uses Docker containers with per-container IP addresses. This project's playground devnet took a lighter approach: `LD_PRELOAD` IP spoofing via a shared library (`libipspoof.so`) that intercepts socket calls and maps each process to a unique fake global IP (e.g. `1.2.3.1` through `1.2.3.255`), supporting up to 255 nodes on a single machine without containerisation. The implications of this constraint for devnet design are discussed further in Section 4.1.

3.3 Stigmerge and the Route-in-DHT Pattern

Stigmerge (Marshall 2024) is a decentralised file-sharing application built on Veilid that demonstrated the coordination patterns this project adapted. The name refers to *stigmergy*,indirect coordination through a shared environment,where peers communicate by reading and writing shared DHT records rather than maintaining direct connections.

Stigmerge established two patterns adopted in this project:

1. **Route-in-DHT**: A node creates a private route, serialises the route blob, and writes it to a DHT subkey. Remote peers read the subkey, import the blob, and send messages via `app_call` to the imported route. This decouples route exchange from direct messaging.
2. **`app_call` for confirmed delivery**: All point-to-point data transfer uses Veilid's `app_call` (request–response/SYN & ACK) rather than `app_message` (fire-and-forget), ensuring that failed deliveries are detected immediately rather than silently dropped.

These patterns were critical for MPC tunnelling, where every message had to be confirmed and route failures had to be detected promptly.

3.4 Auction Privacy in Context

Sako (Sako 2000) was the first to propose a cryptographic auction protocol in which losing bids are never revealed. The scheme uses homomorphic encryption and zero-knowledge proofs so that the auctioneer can publicly verify that the announced winner holds the highest bid, without disclosing the values submitted by any other participant. Sako proved the protocol's correctness and privacy properties formally but did not produce an implementation, leaving practical feasibility as an open question. Blockchain-based auctions remove the trusted auctioneer via smart contracts but expose all bids on-chain and invite front-running (Daian et al. 2019). The Danish sugar-beet auction (Bogetoft et al. 2009) demonstrated MPC in production but relied on three trusted intermediaries operating over direct TCP.

This project combined the privacy guarantees of MPC (non-winning bids never revealed) with the transport-layer anonymity of Veilid (participant IPs never exposed), in a fully decentralised setting with no trusted auctioneer.

The seller occupies a semi-trusted role as MPC party 0. This is justified on two grounds. First, the seller has a direct economic incentive to complete the auction honestly: withholding the decryption key or manipulating results means the sale does not proceed, costing the seller revenue. Second, the seller is the inherent key holder: since they encrypted the listing content, they necessarily possess the decryption key, and any scheme that removes the seller from key delivery requires threshold cryptography (Section 6.4). The seller's power is strictly limited ephemeral route to the winner, not all bids or bidder identities.

Chapter 5 evaluates the cost of this combination in terms of performance overhead, reliability, and residual information leakage.

4 Method

4.1 Development Approach

Development followed an iterative, milestone-driven plan defined in the PDD (Appendix A). The foundation phase (Nov–Dec 2025) focused on understanding the Veilid API, establishing a development environment as well as modifying a draft merge request (Jackson 2024) to fit my usecase (creating hooking a socket library using LD_PRELOAD), and scaffolding the application. Milestone 1 (Jan–Feb 2026) delivered listing and bidding flows over the DHT. Milestone 2 (Feb–Mar 2026) integrated real MPC via MP-SPDZ, culminating in three passing end-to-end tests. Milestone 3 (late Mar 2026) focused on benchmarking, the protocol comparison, and this report.

The original plan included a fourth milestone for Monero testnet settlement; this was descoped to prioritise MPC tunnel reliability and the MASCOT vs. Shamir comparison, which proved more technically demanding and more academically interesting than payment integration.

4.1.1 Infrastructure

As discussed in Section 3.2.2, Veilid requires each node to have a unique public IP address. Developing against the public Veilid network was impractical: misconfigured nodes sharing an IP address trigger an hour-long ban on that IP, during which no Veilid node can bootstrap from it (Rioux 2026a). A private devnet avoids this penalty and allows rapid iteration. The devnet satisfies the unique-IP constraint using two approaches:

The playground devnet launched Veilid processes directly on the host, each assigned a unique IP via `libipspoof.so`, which intercepted socket calls to map fake public addresses (`1.2.3.1`–`1.2.3.255`) to localhost,supporting up to 255 nodes on a single machine. The bootstrap node at `1.2.3.1:5150` was configured following the official Veilid bootstrap guide (Veilid Project 2025a). Lattice nodes took the earliest unused offset above the devnet range, allowing the same addressing scheme to scale across the 40-, 60-, and 80-node devnets used in benchmarking.

Figure 4.1 shows the devnet layout.

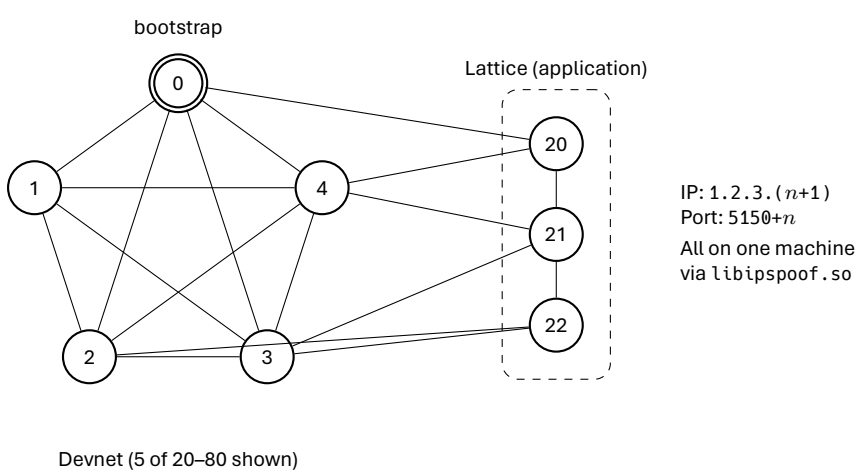


Figure 4.1: Playground devnet topology (subset shown). Node 0 (double border) is the bootstrap server; after initial discovery all nodes form a P2P mesh. Lattice market nodes (shaded) join with offsets above the devnet range. All nodes run on a single machine with `LD_PRELOAD` IP spoofing.

The devnet was automated via Makefile targets. `make demo` built everything from source, started the devnet, and launched a three-node auction with no manual setup. Benchmark runs (Section 5.6) used devnets of 40, 60, and 80 nodes to evaluate how network size affected route stability and auction success rates.

4.2 Architecture

4.2.1 Auction State Machine

Auction logic was abstracted behind traits (`DhtStore`, `RandomSource`) so that the full lifecycle could be tested with mock implementations. Each auction progressed through six phases (Figure 4.2). Phase transitions are driven by events: the deadline timer, DHT reads completing, `app_call` responses arriving, and the MP-SPDZ process exiting.

4.2.2 DHT Record Hierarchy

All marketplace state was stored in Veilid's DHT. Figure 4.3 shows the record hierarchy. Each MPC party published its route blob to its bid's DHT record (subkey 1), the *route-in-DHT* pattern adapted from Stigmerge (Marshall 2024). This aligns with the DHT shared-writer pattern recommended by the Veilid lead developer (Rioux 2025); Veilid 0.6.0 is expected to automate this via DHT route autopublish.

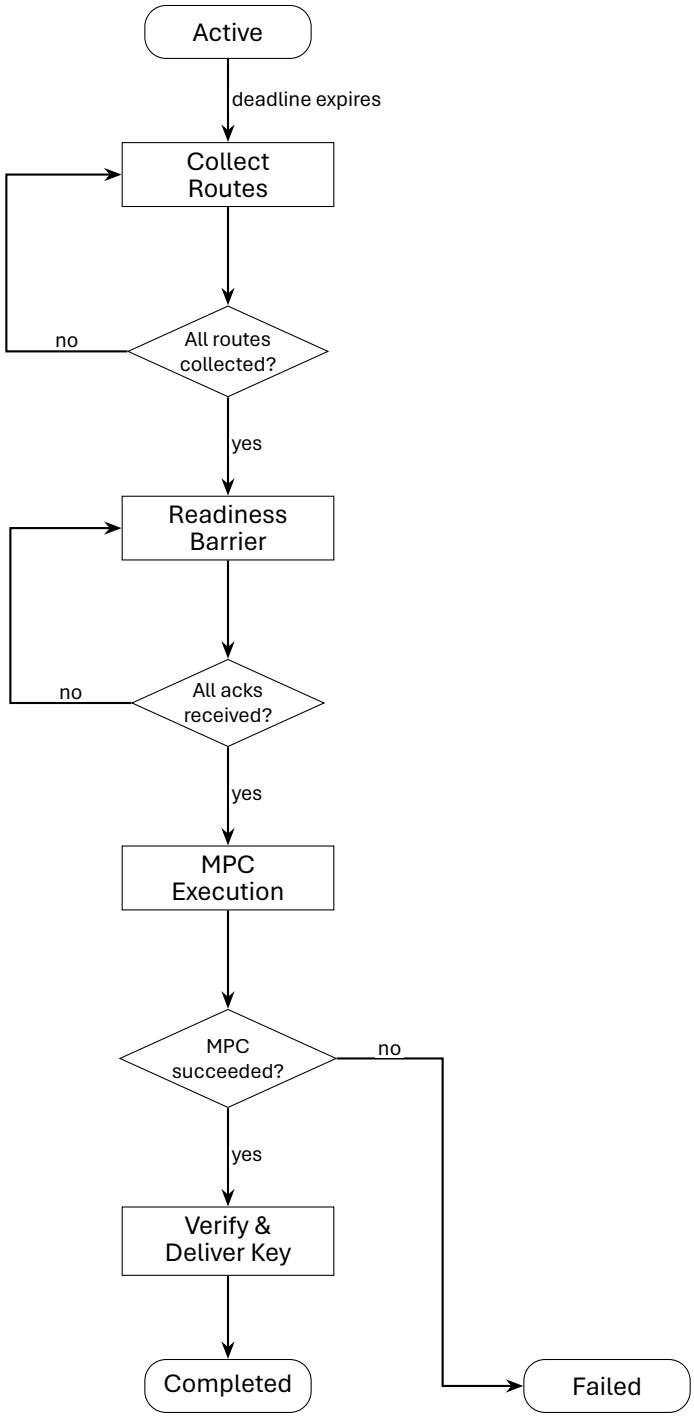


Figure 4.2: Auction phase flowchart. Rectangles are processing phases; diamonds are decision points; rounded boxes are terminal states. The "no" paths from route collection and readiness barrier represent retry loops (DHT re-poll and route refresh respectively). MPC failure leads to the Failed terminal. Each participant independently tracks this flow for every auction.

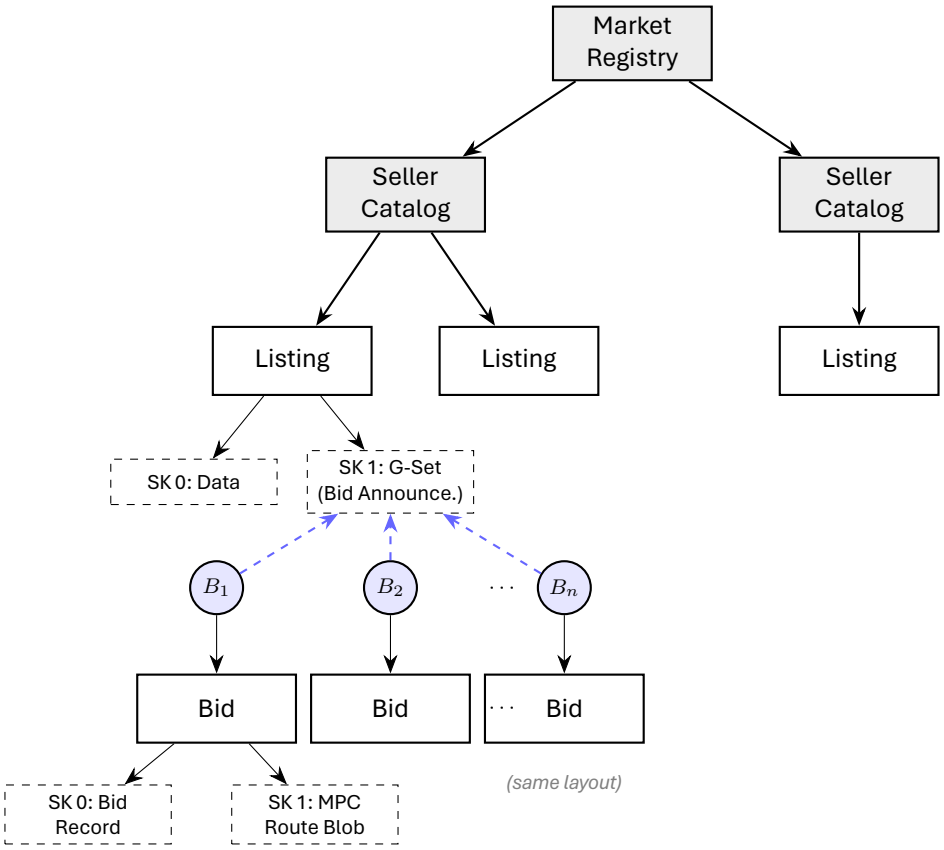


Figure 4.3: DHT record hierarchy. Shaded boxes are directory records; dashed boxes show subkey layout. Bidders (B_1–B_n, dashed arrows) each merge their announcement into the listing's G-Set CRDT (subkey 1) and own an individual bid record. Each node also maintains a local copy of the G-Set in memory; periodic DHT reads merge remote state into the local copy, ensuring convergence. Each bid record's subkey 1 carries the MPC route blob for DHT-backed route exchange.

4.2.3 MPC Tunnel

MP-SPDZ processes communicate via TCP. On a direct network this is trivial; over Veilid, each TCP connection must be tunnelled through onion-routed private routes. The `MpcTunnelProxy` module bridged the two worlds:

1. The MPC orchestrator collects route blobs from all parties via DHT reads (with `app_message` announcement as a fast path and DHT polling as a reliable fallback).
2. A local TCP listener accepts connections from the MP-SPDZ process and forwards each send as a Veilid `app_call` to the target party's private route. The receiver's `app_call` handler feeds the payload into a local TCP connection to its own MP-SPDZ process.
3. Sends are pipelined with a bounded semaphore (depth 8) to overlap network round trips. A per-stream reorder buffer handled out-of-order delivery.

All point-to-point MPC traffic used `app_call` (SYN/ACK) rather than `app_message` (fire-and-forget), following the pattern established by Stigmerge (Marshall 2024). This ensures failed deliveries are detected immediately, a critical requirement for MPC, where a single lost message causes the protocol to hang.

4.2.4 Winner Verification and Content Delivery

After MPC completes, the seller (party 0) knows the winner's identity and winning bid value. Content delivery used a commitment-and-challenge scheme:

1. The seller sends a challenge to the claimed winner via `app_call` over their MPC route.
2. The winner responds with their bid value and nonce, proving knowledge of the committed bid.
3. The seller verifies the commitment $C = \text{SHA-256}(\mathit{bid} \| \mathit{nonce})$ matches the on-DHT record, then sends the AES-256-GCM decryption key.

The PDD originally proposed 1-out-of-N oblivious transfer for this step. The implemented scheme is simpler: because the seller already learns the winning bid as MPC party 0, OT would add complexity without additional privacy benefit. The commitment scheme ensures the seller cannot substitute a different winner, and the `app_call` delivery ensures the key transfer is confirmed.

4.3 Technology Choices

4.3.1 Why Veilid

Veilid provides keyed endpoints, a secure DHT, and private routing in a single embeddable library (Table 3.1). Each auction participant runs a full Veilid node in-process, avoiding the external daemon or SOCKS proxy required by Tor or I2P.

4.3.2 Why MP-SPDZ

MP-SPDZ (Keller 2020) was selected for MPC execution because it supports multiple protocol backends from a single high-level program, enabling the MASCOT vs. Shamir comparison central to the evaluation. The framework is actively maintained, well-documented, and provides the `mascot-party.x` and `shamir-party.x` binaries used in benchmarking.

4.3.3 Why MASCOT and Shamir

These two protocols represent opposite ends of the MPC trust spectrum:

- **MASCOT** (dishonest majority, malicious security): tolerates up to $N-1$ corrupt parties. Suitable for an untrusted P2P setting where any participant might be adversarial. The cost is $O(N^2)$ bandwidth from pairwise OT.
- **Shamir** (honest majority, semi-honest security): requires $< N/2$ corrupt parties. Much lower communication overhead (linear in N), but the assumption may not hold in an open marketplace.

Replicated secret sharing (3-party, honest majority) was used during early development as the simplest protocol to validate the integration pipeline. It is not included in the benchmark comparison because it is fixed to exactly three parties, making scaling analysis impossible.

Other MP-SPDZ backends (semi-honest two-party, BMR garbled circuits, SPDZ2k over rings) were not tested because they either do not support arbitrary N, target a different threat model, or solve the same trust assumption as one of the two selected protocols. MASCOT and Shamir were chosen to isolate the single most impactful variable: honest vs. dishonest majority.

4.3.4 Serialisation Strategy

Two serialisation formats are used:

- **CBOR** for DHT-stored records (listings, bids, route entries). CBOR is self-describing and tolerates schema evolution, important for data that persists in Veilid's DHT (32 KB value limit).
- **Bincode** for ephemeral wire messages (`AuctionMessage` enum via `app_call`). More compact than CBOR but brittle across schema changes, acceptable for messages that are never stored.

4.3.5 UI Framework

Dioxus 0.7.2 provides a React-like desktop UI with a GTK/WebKit backend. The UI was not a focus of the project; it served as a thin presentation layer over the auction logic. A headless binary (`market-headless`) was built for E2E testing, bypassing the UI entirely.

4.4 Testing Strategy

Testing followed a three-tier approach:

1. **Unit tests** ($\sim$169 tests): in `src/` alongside modules. Fast, isolated, no network dependencies.

2. **Integration tests** ($\sim$63 tests): in `tests/`. Use mock DHT, mock time, and mock randomness to simulate full N-party auction flows. A `MultiPartyHarness` sets up N independent `AuctionLogic` instances sharing a `MockDht` backend, enabling deterministic testing of bid ordering, ties, edge cases, and MPC output parsing. All integration tests complete in under one second.

3. **End-to-end tests** (3 tests): in `tests/e2e/`. Run against the playground devnet with `LD_PRELOAD` IP spoofing. Each test launches separate OS processes (`market-headless`) and exercises the full Veilid networking stack. These tests run sequentially (`--test-threads=1`) because they share Veilid infrastructure.

The three E2E tests cover:

- **Happy path**: single auction, 3 parties, listing $\rightarrow$ bids $\rightarrow$ MPC $\rightarrow$ verification $\rightarrow$ decryption key delivery.
- **Sequential auctions**: two auctions in sequence on the same nodes, testing route refresh between sessions.

- **Concurrent auctions**: two auctions with overlapping timelines, testing the constraint that only one MPC session can be active per node (due to Veilid's single-active-route limitation).

4.5 Reuse and Attribution

The project built on three major external components:

- **Veilid** (MPL-2.0): P2P networking, DHT, private routing. Used as a library dependency; no modifications to Veilid source were required for the Lattice application.
- **MP-SPDZ** (BSD-3-Clause): MPC protocol execution. The 46-line `auction_n.mpc` program is original; the framework binaries are used unmodified.

The `app_call` delivery model and route-in-DHT pattern were informed by Stigmerge (Marshall 2024), a Veilid file-sharing application; no code was copied. A full component-level reuse table is provided in Appendix B.

5 Results

This chapter presents the evidence that the system works, how well it performs, and what privacy it actually provides. Each section ties back to the security policy defined in Section 1.2.

5.1 System Architecture

The final system had four layers: a Dioxus desktop UI (with a headless binary for testing), an auction coordination layer (state machine, DHT operations, MPC orchestration), the MPC tunnel proxy (bridging MP-SPDZ's TCP to Veilid `app_call`), and the Veilid overlay (DHT, private routes, node identity). There was no central server and no direct IP connections between participants. All persistent state was CBOR-serialised into DHT records and all real-time communication travelled over onion-encrypted private routes.

5.1.1 DHT as Shared State

The marketplace is bootstrapped from a single shared DHT record: the *master registry*. All Lattice nodes know this record's key (derived from the network key) and share a write keypair, so any node can add entries. The full record hierarchy is shown in Figure 4.3.

The DHT lifecycle (seller registration, listing creation, discovery, bidding, and MPC route exchange) follows the record hierarchy in Figure 4.3. The master registry and bid announcement registries use a G-Set CRDT: each write merges the local set with the DHT value via commutative union, so all nodes converge regardless of write order.

5.2 Auction Protocol

The auction lifecycle, following the state machine in Figure 4.2, proceeds in seven phases:

1. **Listing creation.** The seller generates an AES-256-GCM key, encrypts the listing content, publishes the listing record to the DHT, and announces the listing key via broadcast `app_message` to registered peers.

2. **Bid submission.** Each bidder creates a commitment $C = \text{SHA-256}(\textit{bid} \| \textit{nonce})$ with a cryptographically random 32-byte nonce, writes a bid record to the DHT, and announces the bid key to the listing's announcement registry. The seller's node automatically places an auto-bid at the reserve price (party 0).

3. **Deadline and party assignment.** When the listing deadline expires, all nodes independently compute the same party assignment by sorting bidders by timestamp ascending (the seller's auto-bid is always earliest, becoming party 0). This deterministic assignment requires no coordination.

4. **MPC route exchange.** Each party creates a Veilid private route and writes the route blob to their bid record's subkey 1. Parties discover each other's routes via DHT reads with `app_message` announcements as a fast path. A readiness barrier (`MpcReady` via `app_call`) ensures all routes are live before proceeding.

5. **MPC execution.** The `MpcOrchestrator` launches a local `mascot-party.x` (or `shamir-party.x`) process, feeds each party's bid value as secret input, and tunnels all inter-party TCP traffic through the Veilid `app_call` tunnel. The MPC program (Listing 5.1) computes the maximum bid and reveals the winner ID and winning bid value only to party 0.

6. **Winner verification.** The seller sends a challenge to the claimed winner via `app_call`. The winner responds with their bid value and nonce; the seller verifies that SHA-256$(bid \| nonce) = C$.

7. **Decryption key transfer.** Upon successful verification, the seller sends the AES-256-GCM decryption key to the winner via `app_call`.

```
1  # n-Party Sealed-Bid Auction (First-Price)
2  # Party 0 = seller (reserve price). Parties 1..N = bidders.
3
4  n_parties = int(program.args[1])
5
6  # Each party inputs their bid as a secret integer
7  bids = Array(n_parties, sint)
8  for i in range(n_parties):
9      bids[i] = sint.get_input_from(i)
10
11 # Find maximum bid and winner
12 max_bid = bids[0]
13 winner = sint(0)
14 for i in range(1, n_parties):
15     is_higher = bids[i] > max_bid
16     max_bid = is_higher.if_else(bids[i], max_bid)
17     winner = is_higher.if_else(sint(i), winner)
18
19 # Reveal winner and bid ONLY to party 0 (seller)
20 winner_id = winner.reveal_to(0)
21 winning_bid = max_bid.reveal_to(0)
22 print_ln_to(0, 'Winner:␣Party␣%s', winner_id)
23 print_ln_to(0, 'Winning␣bid:␣%s', winning_bid)
24
25 # Each other party learns only won/lost
26 for party in range(n_parties):
27     am_i_winner = (winner == sint(party)).reveal_to(party)
28     print_ln_to(party, 'You␣won:␣%s', am_i_winner)
```

Figure 5.1: The MPC auction program (`auction_n.mpc`), written in the MP-SPDZ DSL. The comparison on line 16 and conditional select on lines 17–18 are computed over secret-shared values - $sint$, the provided primitive for an (appropriately named) secret int within SPDZ. No party sees intermediate results. The `reveal_to(0)` calls on lines 21–22 ensure that only party 0 (the seller) learns the winner identity and winning bid.

5.3 MPC Integration

5.3.1 The Auction Program

The `auction_n.mpc` program (46 lines) implemented a first-price sealed-bid auction for N parties. Each party inputs their bid as a secret integer; the program computes the maximum and reveals (i) the winner identity and winning bid to party 0 only, and (ii) a binary "won: 0/1" to each other party. The program is compiled once and executed with the appropriate party count flag (`-N`). Note that for honest majority protocols executed within SPDZ, OpenSSL certificates are required to be generated at compile time for each potential participant, and shared before the MPC reaches runtime. This is an advantage of using a dishonest majority protocol; no SSL secret shares prior to executing the MPC.

5.3.2 Tunnel Architecture

The MPC tunnel sent TCP over Veilid private routes. Outgoing and incoming traffic each ran in its own async task. Sends were pipelined (up to eight `app_call` requests in flight at once) to minimize network latency. The receiver reassembled the byte stream using sequence numbers and a reorder buffer, since Veilid does not guarantee ordered delivery. Veilid has a limit of 32KB payloads per communication envelope, and so the app_calls were maxsized at 32KB each.

Table 5.1 shows the data volumes per protocol and party count. MASCOT's pairwise OT produces two to three orders of magnitude more tunnel traffic than Shamir, meaning more `app_call` round trips and longer wall-clock times.

Protocol	N	Per-party (MB)	Total (MB)	App_calls
MASCOT	3	37	107	3,416
MASCOT	6	209	1,192	38,165
MASCOT	10	653	6,213	198,747
Shamir	3	0.02	0.11	4
Shamir	6	0.21	1.2	39
Shamir	10	0.61	4.2	134
Shamir	20	2.5	25.6	842

Table 5.1: Data volumes tunnelled through Veilid per auction. Each `app_call` carries up to 32 KB of payload. MASCOT transfers roughly 1,000$\times$ more data than Shamir at $N{=}3$, growing to 1,500$\times$ at $N{=}10$ due to $O(N^2)$ pairwise OT.

5.3.3 Route Lifecycle Challenges

Veilid private routes are the single largest source of engineering complexity in the system. Three problems were encountered and solved:

1. **Silent death.** Routes die without notification; the only signal is a timeout on the next `app_call`. The system implemented reactive route refresh: if the readiness barrier detected fewer successful acknowledgements than expected, the node recreated its route, republished to the DHT, and polled peers' DHT entries for updated blobs.

2. **Single active route.** Veilid supports only one active private route per node. Creating a second route invalidates the first. This means MPC sessions must be sequential, never concurrent. When multiple auctions expire simultaneously, expired listings are sorted deterministically by key to ensure all nodes process them in the same order.

3. **Self-import requirement.** After creating a private route via `new_private_route()`, the node must call `import_remote_private_route()` on its own blob. Without this step, only one of N created routes is deliverable. This behaviour is undocumented but was confirmed by the Veilid core developers.

These workarounds add approximately 200 lines of code. Veilid 0.6.0 will introduce DHT-published route blobs that are automatically fetched and imported on use, eliminating the need for manual route management (Rioux 2025).

5.4 User Interface

The Dioxus desktop application provided four main views: a marketplace browser showing active listings, a listing creation form with deadline and reserve price fields, a bid placement dialog, and an auction results view showing the winner notification or decryption key. The UI communicated with the auction coordination layer through shared `Arc<Mutex<AppState>>` state, polled by the Dioxus render loop.

The UI was not a focus of the project; it served as a thin presentation layer demonstrating that the underlying protocol is usable interactively. A separate headless binary (`market-headless`) provided the same auction logic without the UI, enabling automated E2E testing and benchmarking. The headless binary accepts listing parameters, bid values, and deadlines via command-line arguments and environment variables, writes structured output (JSON events) to stdout, and exits after the auction completes or times out.

5.5 Testing and Verification

5.5.1 Test Coverage

Table 5.2 summarises the test suite.

Level	Scope	Count	Runtime
Unit	Modules (traits, serialisation, crypto)	169	< 1 s
Integration	Full N-party auctions (mock DHT)	63	< 1 s
E2E	Playground devnet, real Veilid networking	3	3–8 min each

Table 5.2: Test matrix.

5.5.2 Integration Test Results

The mock-based integration tests exercise the auction state machine with 3, 5, and 10 parties, covering:

- Highest bid wins and receives decryption key.
- Tied bids resolved by earliest timestamp.
- Reserve price enforced (auto-bid wins if no bidder exceeds it).
- Expired listings detected and processed.
- Commitment verification rejects incorrect nonces.

All 232 tests (unit + integration) pass consistently with zero flakiness.

5.5.3 End-to-End Test Results

The three E2E tests ran against a 20-node playground devnet:

- **Happy path** (169 s): single 3-party auction completes successfully, winner receives decryption key.
- **Sequential** (380 s): two auctions in sequence; tests route refresh between sessions.
- **Concurrent** (284 s): two auctions with overlapping deadlines; verifies deterministic ordering prevents deadlock.

All three tests pass reliably on a clean devnet. The devnet must be restarted between test runs because Veilid's routing tables retain stale entries for nodes that no longer exist; reconnecting on the same IP triggers a one-hour penalty before the node is accepted again. Additionally, reconnecting with a different node identity on the same IP forces peers to flush their cached knowledge of the previous node, and Veilid nodes generally reject multiple node identities from a single IP address.

5.6 Performance Benchmarks

The benchmark harness ran each configuration 5 times across three devnet sizes (40, 60, 80 nodes), with party counts from 3 to 20. Benchmarks were run across two machines and results were combined:

- **Laptop**: AMD Ryzen AI 5 340 (12 cores) @ 4.90 GHz, 32 GB DDR5, Arch Linux.
- **Desktop**: AMD Ryzen 7 5800X (16 threads) @ 4.85 GHz, 32 GB DDR4, Arch Linux.

Figures in this section are generated from 354 trial rows (153 direct TCP, 201 over Veilid).

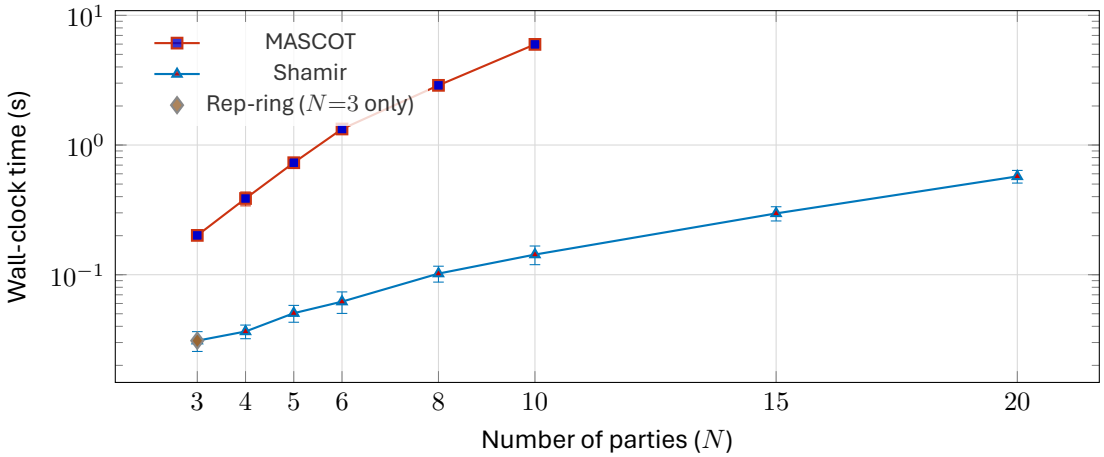


Figure 5.2: Direct MPC computation time (localhost TCP, no Veilid overlay). MASCOT scales quadratically with party count due to pairwise OT setup; Shamir scales linearly. Error bars show ± 1 standard deviation over 10 iterations.

5.6.1 Direct MPC Baseline

Figure 5.2 shows MPC computation time on direct TCP (localhost, no Veilid overlay). MASCOT scales quadratically with party count due to pairwise OT setup: 0.21 s at $N{=}3$, rising to 5.83 s at $N{=}10$. Shamir scales linearly: 0.03 s at $N{=}3$ to 0.58 s at $N{=}20$. The performance difference is roughly $30\times$ at $N{=}10$.

Figure 5.7 shows the bandwidth cost: MASCOT sends 37 MB per party at $N{=}3$ and 653 MB at $N{=}10$ ($O(N^2)$ growth), while Shamir sends 0.02 MB at $N{=}3$ and 2.5 MB at $N{=}20$ (linear growth).

5.6.2 Veilid Tunnel Overhead

Figure 5.6 compares direct TCP with Veilid-tunnelled MPC computation time. The Veilid tunnel added approximately $15{-}19\times$ overhead for MASCOT and $10{-}30\times$ for Shamir. This section analyses where that overhead comes from.

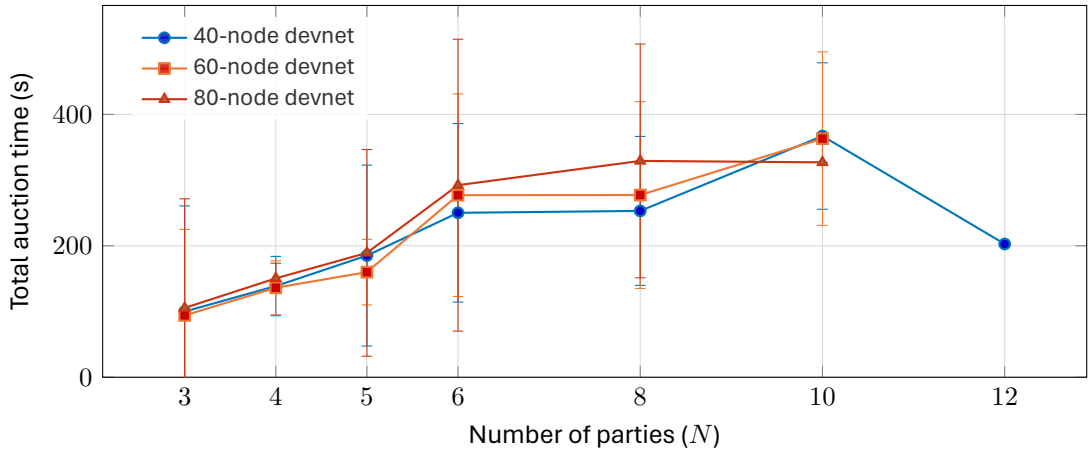


Figure 5.3: Total auction time over Veilid using MASCOT (median of successful runs, $\pm 1\sigma$). Route exchange and MPC tunnel latency dominate. Larger devnets reduce success rates (see Figure 5.8) but successful runs show similar wall-clock times.

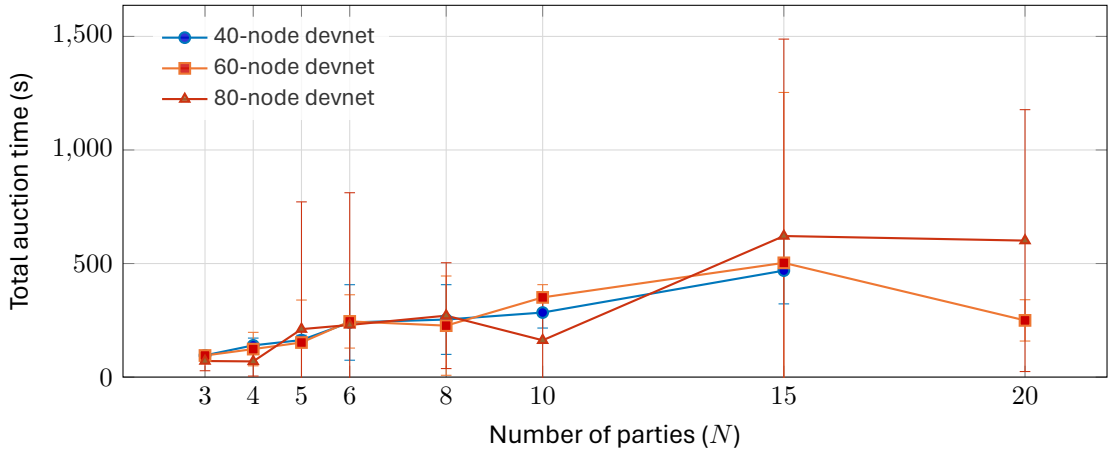


Figure 5.4: Total auction time over Veilid using Shamir secret sharing (median of successful runs, $\pm 1\sigma$). The 40-node devnet has insufficient relay capacity to maintain routes for all 20 parties ($N{=}20$: 0/10 success). Only successful iterations are included; timeouts are excluded.

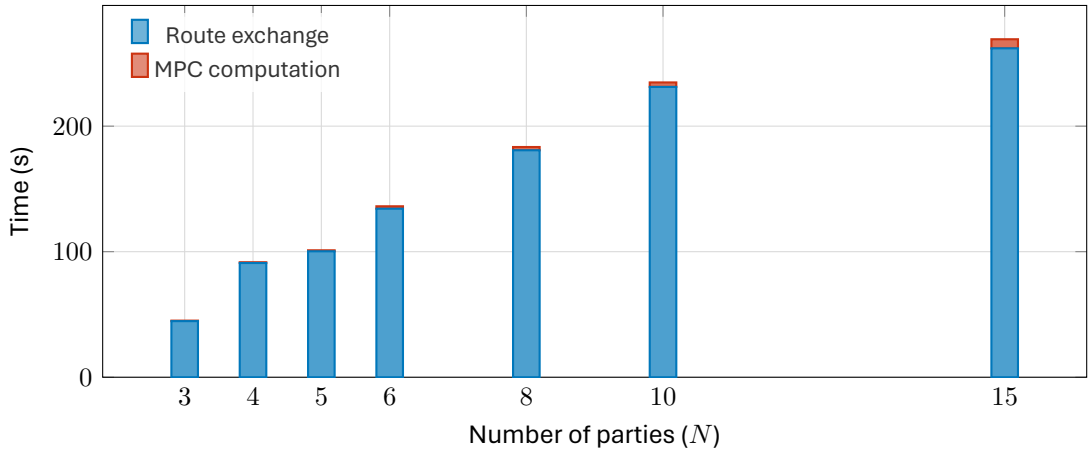


Figure 5.5: Time breakdown for Shamir auctions on a 40-node devnet. Route exchange through Veilid private routes dominates total time; the MPC computation itself (tunnelled over Veilid `app_call`) accounts for less than 10% at most party counts.

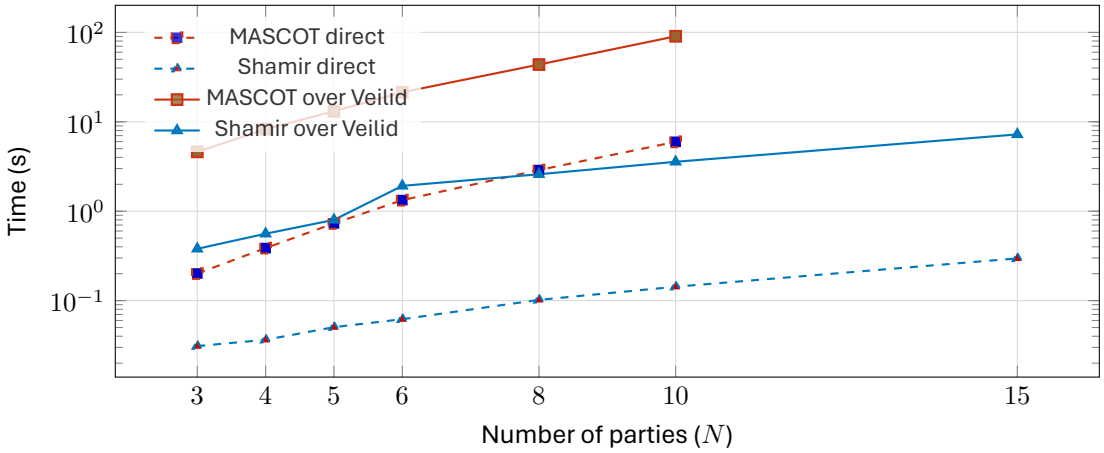


Figure 5.6: MPC computation time: direct TCP vs. tunnelled through Veilid private routes (40-node devnet, median of successful runs). The Veilid tunnel adds approximately $15{-}19\times$ overhead for MASCOT and $10{-}30\times$ for Shamir, attributable to per-round `app_call` latency through onion-routed hops.

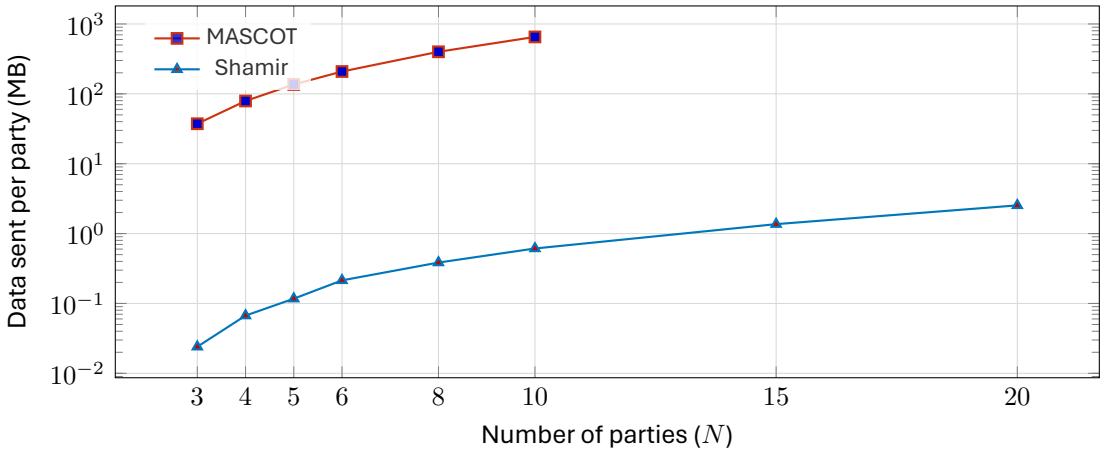


Figure 5.7: Per-party data sent during MPC computation (direct TCP). MASCOT's pairwise oblivious transfer requires $O(N^2)$ bandwidth growth (37 MB at $N{=}3$ to 653 MB at $N{=}10$), while Shamir's secret-sharing approach grows linearly (0.02 MB at $N{=}3$ to 1.4 MB at $N{=}20$). MASCOT benchmarks beyond $N{=}12$ were not feasible: the host desktop environment ran out of memory under the combined load of devnet nodes and MPC processes.

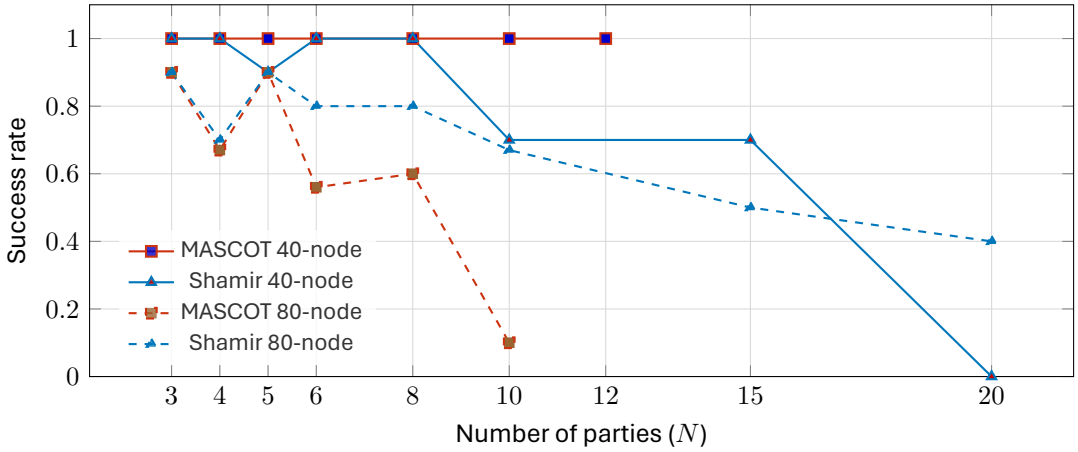


Figure 5.8: Success rate by party count and devnet size. MASCOT maintained 100% success on the 40-node devnet up to $N{=}12$ but degraded on larger devnets. Shamir scaled further in party count but dropped to 0% at $N{=}20$ on 40 nodes.

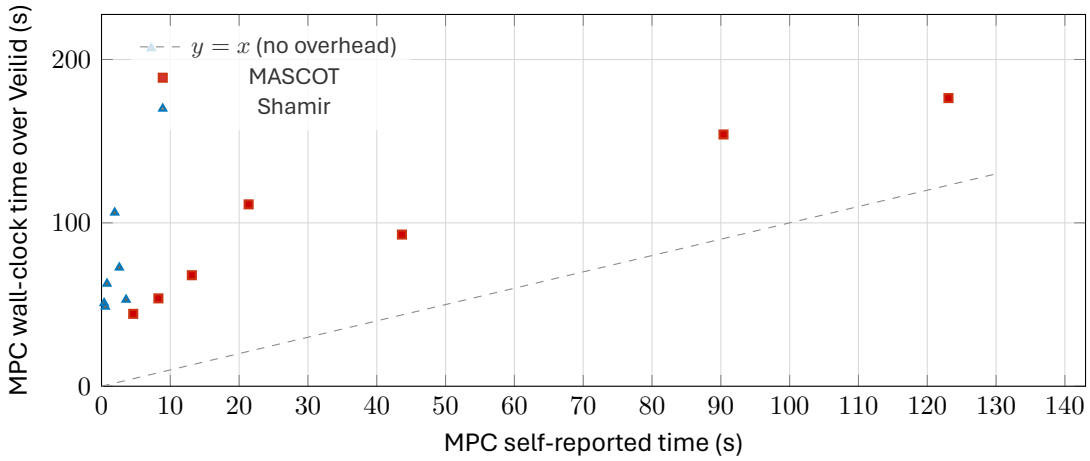


Figure 5.9: MPC computation time vs. total non-route time (40-node devnet). Points above the $y{=}x$ line indicate overhead from Veilid tunnel latency. MASCOT incurs larger absolute overhead due to its higher round count.

Per-round latency. Each MPC communication round requires at least one `app_call` round trip through Veilid's onion-routed private routes. Each private route traverses two relay hops (the default on the devnet), so a single `app_call` traverses at minimum four hops (sender $\rightarrow$ two relays $\rightarrow$ recipient, then back). At each hop, Veilid performs Curve25519/XSalsa20-Poly1305 decryption on the envelope, routes the payload to the next hop, and re-encrypts. Table 5.3 shows the resulting per-round latency for both protocols.

Protocol	N	Rounds	Direct (ms/round)	Veilid (ms/round)	Slowdown
MASCOT	3	257	0.65	17.9	$28\times$
MASCOT	6	963	1.28	22.2	$17\times$
MASCOT	10	2483	2.32	36.4	$16\times$
Shamir	3	35	0.05	10.9	$209\times$
Shamir	10	122	0.38	29.3	$77\times$
Shamir	20	242	1.26	—	—

Table 5.3: Per-round MPC latency: direct TCP vs. Veilid tunnel (40-node devnet). Per-round latency is computed as MPC wall-clock time divided by round count. Shamir $N{=}20$ is absent because no trial succeeded on the 40-node devnet.

The per-round overhead explains why the *aggregate* overhead factor differs between MASCOT and Shamir. MASCOT performs substantial local computation per round (OT extension, MAC generation), so the fixed $\sim$18 ms network latency is a smaller fraction of each round's total time. Shamir's local computation is sub-microsecond, so the same $\sim$18 ms network latency accounts for nearly all of each round's cost, producing a higher per-round slowdown factor. In absolute terms, Shamir remains far faster due to fewer rounds and less data per round.

Tunnel throughput. The tunnel's effective throughput is limited by `app_call` round-trip time and pipeline depth. With a pipeline depth of 8 and ∼18 ms per `app_call`, the theoretical maximum throughput is 8×32 KB$/18$ ms ≈ 14 MB/s. In practice, the tunnel sustained approximately 2.5 MB/s for MASCOT $N{=}3$ (109 MB in 44 s of MPC time) and 3 KB/s for Shamir $N{=}3$ (116 KB in 0.38 s). The gap between theoretical and observed throughput for MASCOT arises because not all 8 pipeline slots are always filled: the MP-SPDZ process alternates between sending data and waiting for responses in each round.

Cryptographic cost per hop. Each Veilid relay hop performs envelope decryption (XSalsa20-Poly1305) and re-encryption for the next hop. On the benchmark hardware (Ryzen AI 5 340), XSalsa20-Poly1305 processes ∼1 GB/s, so the per-hop crypto cost for a 32 KB `app_call` payload is ∼32 µs. This is negligible compared to the network round-trip latency. The overhead is therefore dominated by *network traversal* (hop-by-hop forwarding, connection multiplexing, congestion backoff), not by cryptographic computation.

5.6.3 Total Auction Time

Figures 5.3 and 5.4 show total end-to-end auction time (route exchange + MPC + verification). Route exchange dominated total time at every party count. Figure 5.5 shows that for Shamir on a 40-node devnet, MPC computation accounted for less than 10% of wall-clock time at all party counts tested. Even for MASCOT, where the tunnel carries hundreds of megabytes, route exchange still accounted for 40–65% of total time.

The route exchange phase consists of four sequential steps, each involving Veilid operations:

1. **Route construction** (∼5–15 s): each party calls `new_custom_private_route()` and self-imports the blob. This involves Diffie–Hellman key exchanges with two relay nodes.
2. **DHT publishing** (∼2–5 s): the route blob is written to the party's bid DHT record (subkey 1). DHT writes propagate through the overlay's Kademlia routing.
3. **Route discovery** (∼10–40 s, scales with N): each party reads all other parties' DHT records to collect route blobs. The $N{-}1$ DHT reads are sequential because each read forces a network round trip to the DHT record's close nodes.
4. **Readiness barrier** (∼5–30 s): each party sends an `MpcReady` message via `app_call` to every other party's imported route, confirming all routes are live. Failed deliveries trigger a reactive refresh cycle: recreate the local route, republish to DHT, re-poll peers.

Route discovery dominates because it requires $O(N)$ sequential DHT reads, each taking 1–4 s depending on how quickly the DHT close nodes respond. At $N{=}10$, nine DHT reads at ∼3 s each account for ∼27 s of the 213 s median route exchange time; the remainder is consumed by the readiness barrier and retry cycles.

This analysis suggests that reducing route exchange time (through persistent routes, DHT caching, or parallel route discovery) would have a larger impact on total auction time than optimising the MPC tunnel itself.

5.6.4 Reliability and Success Rates

Not all trials succeeded. Figure 5.8 shows success rates by party count and devnet size. On the 40-node devnet, MASCOT achieved 100% success up to $N{=}12$ (the largest party count tested). Shamir maintained high success rates up to $N{=}10$ (70%) but dropped to 0% at $N{=}20$. On the 80-node devnet, both protocols degraded: MASCOT fell to 10% at $N{=}10$ and Shamir to 40% at $N{=}20$.

Three distinct failure modes were observed:

1. **Route construction timeout**: the node could not find a suitable relay node to form a 1-hop private route within the 50 s deadline (Veilid defaults to 2 hops; this was reduced to 1 for the smaller devnet topology). This occurred more frequently on larger devnets where routing table convergence took longer.
2. **Dead route during MPC**: a relay node in an active private route became unreachable mid-computation. Because Veilid does not notify applications of route death, the tunnel detected this only after a 10 s `app_call` timeout, at which point the MPC process had already stalled.
3. **MPC process timeout**: the 600 s global deadline was exceeded, typically because multiple dead-route recovery cycles consumed the time budget.

All failures were attributable to Veilid route instability under load, not to bugs in the auction protocol or MPC program. The MP-SPDZ processes themselves never produced incorrect results when they completed.

5.6.5 Devnet Size Effects

Larger devnets (60, 80 nodes) showed *worse* performance, not better. This is counterintuitive: one might expect more relay nodes to provide more routing options and thus better availability. Three factors explain the degradation:

1. **Routing table convergence.** Veilid's Kademlia-based routing tables grow with network size. On a freshly started 80-node devnet, each node must discover and validate up to 79 peers via iterative lookups. During this convergence period, route construction attempts may select relay nodes that have not yet stabilised their own routing tables.
2. **Resource contention.** All devnet nodes ran on the same physical machine, sharing CPU, memory, and network stack. At 80 nodes, each Veilid process consumed ∼50 MB of RAM and maintained ∼40 UDP connections, totalling ∼4 GB RAM and ∼3,200 sockets. Context switching and socket buffer contention added artificial latency not present in a real distributed deployment.
3. **Relay selection quality.** Veilid selects relay nodes based on latency and reliability metrics that take time to accumulate. On larger devnets, the probability of selecting an under-performing relay increases during the initial convergence period.

On the public Veilid network, with thousands of nodes across diverse hardware and with stable long-running relays, the behaviour would likely differ. However, public network testing was not practical: Veilid enforces one-hour bans per IP for misconfigured nodes (Rioux 2026a), which occurs frequently during development, and running behind CGNAT meant nodes did not meet Veilid's reachability requirements for full relay participation, motivating the devnet approach.

5.6.6 Hardware and Software Limits

The practical limit for MASCOT over Veilid on this devnet is approximately $N{=}6$ (100% success rate on the 40-node devnet, degrading on larger devnets). Beyond this, the $O(N^2)$ bandwidth overwhelms the tunnel: at $N{=}10$, MASCOT sends ∼637 MB per party through the tunnel, requiring ∼2,500 `app_call` round trips.

Shamir scales further because its linear bandwidth means the tunnel carries ∼2.3 MB rather than 637 MB at comparable party counts. Shamir $N{=}15$ achieved 70% success on the 40-node devnet; $N{=}20$ succeeded on 60 and 80-node devnets (30–40%) but not on 40 nodes, likely due to insufficient relay diversity at the smaller size.

The bottleneck is not CPU or memory but *Veilid route latency*. Table 5.4 illustrates the difference between warm and cold devnets.

Condition	MPC time	Route time	Total	ms/round
40-node devnet (median)	4.61 s	55.66 s	99.97 s	17.9
Direct TCP (no Veilid)	0.20 s	—	0.20 s	0.78

Table 5.4: MASCOT $N{=}3$: Veilid tunnel vs. direct TCP (40-node devnet, 10 successful trials). The tunnel adds 17.1 ms/round of overhead, dominated by Veilid relay latency.

The Veilid tunnel adds ∼18 ms of network round-trip time per `app_call`, compared to sub-millisecond on localhost TCP.

5.7 Security Analysis

This section evaluates the system against the security policy from Section 1.2, identifying what is protected, what leaks, and the severity of each residual channel.

5.7.1 Threat Model

Four adversary types are considered:

- **Honest-but-curious bidder**: follows the protocol but attempts to learn other bids from observable data.
- **Malicious seller**: controls the listing, decryption key, and MPC party 0 output.
- **Passive network observer**: monitors DHT writes and Veilid message patterns without participating.
- **Colluding parties**: multiple participants share information to break the MPC threshold.

5.7.2 Timing Side Channel on Party Assignment

Party order is determined by bid timestamp. A network observer monitoring DHT writes can observe bid timing, inferring party assignments. If the adversary knows the mapping between party IDs and bidders, they can correlate MPC outputs with specific participants.

Severity: Medium. **Mitigation**: None currently. Possible fixes include random party assignment (requiring a pre-committed randomness source) or padding bid submissions to fixed intervals.

5.7.3 Seller as Single Point of Trust

The seller (party 0) sees the winner identity and winning bid value, controls decryption key release, and can refuse verification (griefing) or collude with a preferred bidder. This is the most significant trust assumption in the system.

Compared to the Danish sugar-beet auction (Bogetoft et al. 2009), where the auctioneer was a trusted cooperative, this system gives the seller *less* power (they see only the winning bid, not all bids) but *more* ability to grief (they can withhold the key).

Severity: High (by design). **Mitigation**: Threshold decryption (key shared among MPC parties) would remove this assumption; see Section 6.4.

5.7.4 Winner Identity Linkage

The challenge-response protocol reveals the winner to the seller before the decryption key is sent. This is necessary for protocol correctness: the seller must verify the bid commitment before releasing the key.

Severity: Medium (by design). The PDD proposed 1-out-of-N oblivious transfer to avoid this linkage; however, since the seller already learns the winner as MPC party 0, OT would not provide additional privacy.

5.7.5 DHT Metadata Leakage

Bid record keys are publicly announced via the listing's announcement registry. The *number* and *timing* of bids are observable by any DHT reader, even though bid *values* are hidden behind commitments. This is a form of participation metadata leakage.

Severity: Low. **Partial mitigation**: Veilid's private routing reduces IP-level correlation between a bid record write and a specific network identity.

5.7.6 MPC Route Correlation

MPC route blobs are written to bid record subkeys before MPC execution. A network observer could correlate route creation timing with auction participation, partially deanonymising MPC parties at the network layer.

Severity: Medium. **Partial mitigation**: Veilid's onion routing limits direct observation (default hop count = 2; benchmarks used hop count = 1 for performance). However, the anonymity set is small on a devnet.

5.7.7 Small-N Security Considerations

The choice between MASCOT and Shamir has direct security implications at small party counts:

- **MASCOT** ($N{=}3$): tolerates up to 2 corrupt parties. Even with 2 colluding bidders, the protocol remains secure,only the colluding parties' own inputs are "lost" (they already know their own bids).
- **Shamir** ($N{=}3$): requires < 2 corrupt parties (i.e., at most 1). A single colluding bidder plus the seller can reconstruct all shares, completely breaking privacy.

For small auctions (the common case in a marketplace), MASCOT's stronger security guarantee is significant, despite its higher computational cost.

5.7.8 Summary of Side Channels

Table 5.5 summarises the identified information leakage channels.

Side channel	Severity	Mitigated?	Exploitable by
Bid timing $\rightarrow$ party ID	Medium	No	Network observer
Seller sees winner + bid	High	By design	Malicious seller
Winner identity linkage	Medium	By design	Seller
DHT participation metadata	Low	Partially	Any DHT reader
MPC route correlation	Medium	Partially	Network observer
Shamir threshold ($N{=}3$)	High	Use MASCOT	Colluding parties

Table 5.5: Summary of residual information leakage channels.

6 Conclusions and Discussion

6.1 Objectives Revisited

1. **Listings and purchases: achieved.** The system implemented the full auction lifecycle: listing creation with AES-256-GCM content encryption, sealed-bid submission with SHA-256 commitments, deadline-based expiry, and decryption key delivery to the verified winner. This is demonstrated by the integration test suite (232 passing tests) and three end-to-end tests on the playground devnet.

2. **Sealed-bid via MPC: achieved.** The MASCOT and Shamir protocols both computed the correct winner without revealing non-winning bid values. The 46-line `auction_n.mpc` program ran correctly for party counts from 2 to 20 (Shamir) or 2 to 10 (MASCOT, limited by Veilid tunnel throughput).

3. **MPC-gated decryption: achieved (modified design).** The PDD proposed 1-out-of-N oblivious transfer for decryption key delivery. The implemented design used a simpler commitment-and-challenge scheme: the winner proved knowledge of their committed bid, and the seller released the key upon verification. As discussed in Section 4.2.4, this provided equivalent privacy because the seller already learns the winning bid as MPC party 0.

4. **Settlement on Monero testnet: not implemented.** This objective was descoped as it was not inline with the research goals of the project, being more suited to a production system. The time originally allocated was spent on MPC tunnel reliability and the MASCOT vs. Shamir benchmark comparison.

9

6.2 Discussion

The central question was whether MPC can be performed over a privacy-preserving peer-to-peer overlay at all. The answer is yes: MASCOT and Shamir both produced correct auction results when tunnelled through Veilid's onion-routed private routes, for party counts from 2 to 20. No trusted intermediary was required, no participant's IP address was exposed, and non-winning bid values were never revealed to anyone.

MPC over a privacy overlay is feasible. The Danish sugar-beet auction (Bogetoft et al. 2009) demonstrated MPC for auctions, but over direct TCP among a small set of trusted servers. This project showed that the same class of computation works in a fully decentralised, anonymous setting. The MPC tunnel sustained the hundreds of megabytes of traffic that MASCOT requires (109 MB at $N{=}3$, 6.2 GB at $N{=}10$) and delivered correct results at 100% success rate up to $N{=}12$ on the 40-node devnet.

The cost of privacy is quantifiable. Veilid's onion routing added $15{-}19\times$ overhead to MASCOT computation time and $10{-}30\times$ to Shamir, attributable to $\sim$18 ms per relay hop. This is the fundamental cost of routing each MPC round through multiple encrypted hops rather than direct TCP, and it is the price paid for hiding participants' network identities.

Route management overhead is separable and fixable. Route exchange dominated total auction time (over 70% of wall-clock duration), but this is an artefact of Veilid's current single-active-route constraint, which forces routes to be created and exchanged fresh for every auction. With persistent routes (planned for Veilid 0.6.0 (Rioux 2026b)), this cost would be amortised to near zero, reducing a 100 s auction to $\sim$15 s.

The security model involves real tradeoffs. MASCOT provides stronger guarantees (dishonest majority) but at a bandwidth cost that limits practical party counts. Shamir is dramatically cheaper but relies on an honest-majority assumption that may not hold in an open marketplace. For small auctions ($N{=}3$), MASCOT's stronger security is particularly valuable because a single corrupt party under Shamir breaks all privacy.

6.3 Legal, Social, Ethical and Professional Issues

Dual-use risk. A private marketplace could in principle facilitate illicit exchange. This risk is mitigated by design: the system operates only on a local devnet, no deployment scripts target the public Veilid network, no real funds are handled, and no mainnet or testnet cryptocurrency wallets are integrated. The project is categorised as a Category 1 prototype not intended for distribution (PDD Section 1.6).

Privacy vs. accountability. Sealed bids protect bidders but complicate dispute resolution. In a production deployment, a reputation system or escrow mechanism would be needed to balance privacy with accountability.

Licensing compliance. Veilid is licensed under MPL-2.0 (file-level copyleft); all modifications to Veilid files are published under MPL. MP-SPDZ is BSD-3-Clause (permissive). All other dependencies are MIT or Apache-2.0. No licensing conflicts exist.

Professional conduct. No human participants were involved. All testing used automated processes on local infrastructure. The BCS Code of Conduct was followed: the work does not endanger individuals, and the dual-use risks are acknowledged and mitigated.

Statement on illicit uses. The author explicitly condemns any attempt to adapt this research for illicit trade, exploitation, or other illegal activity. The work is motivated by technical research questions: privacy-preserving auction protocols, MPC integration, and overlay networking,rather than any intent to facilitate wrongdoing.

6.4 Future Work

- **Threshold decryption.** Share the decryption key among MPC parties using threshold cryptography, removing the seller-as-single-point-of-trust assumption. This is the highest-priority improvement.
- **Vickrey (second-price) auctions.** The winner pays the second-highest bid rather than their own. This would require the MPC program to output the second-highest value and changes to the verification scheme, since the winner would need to prove they bid *at least* the second price rather than proving their exact bid.
- **Monero testnet settlement.** The originally planned payment layer would complete the end-to-end flow from bidding to settlement.
- **Anti-replay nonces.** Add explicit nonces to `DecryptionHashTransfer` messages to prevent replay attacks across auctions.

- **Formal verification.** Model the protocol in a formal verification language to mechanically prove the privacy properties claimed in the security analysis.
- **Public network evaluation.** Benchmark on the public Veilid network to understand real-world route latency and anonymity set size.
- **Adaptive protocol selection.** Automatically choose between dishonest-majority (MASCOT) and honest-majority (Shamir) based on party count: at small N, corrupting a majority is realistic and MASCOT's stronger guarantees justify the cost; as N grows, the difficulty of corruption increases and the system could safely switch to Shamir for better scalability.

6.5 Reflections

The project's primary contribution is a proof of feasibility: multi-party computation can be performed over a privacy-preserving overlay network, and the cost of doing so can be measured. Before this work, no empirical data existed on the overhead of tunnelling MPC through onion-routed messaging. The benchmarks confirmed that the honest-vs-dishonest-majority choice has first-order effects on both performance and reliability, not just theoretical security. This is consistent with findings from Araki et al. (Araki et al. 2017), who demonstrated that honest-majority protocols scale orders of magnitude better than dishonest-majority alternatives. This suggests a pragmatic hybrid strategy: at small party counts, where corrupting a majority is realistic, dishonest-majority protocols like MASCOT are worth their cost. As the party count grows, the difficulty of corrupting a majority increases, and the performance advantage of honest-majority protocols becomes compelling. At some threshold, the threat model can safely be relaxed in exchange for dramatically better scalability.

The most important distinction that emerged is between *fundamental* and *accidental* overhead. The MPC tunnel's 15–19$\times$ slowdown is fundamental: it is the cost of onion routing each round through encrypted relay hops, and it buys real privacy. The route exchange overhead (50–90 s per auction) is accidental: it results from Veilid's single-active-route constraint forcing route teardown and recreation for every auction. With persistent routes, this cost disappears entirely, and the system becomes practical for interactive use ($\sim$15 s per auction at $N{=}3$).

The most challenging aspect was Veilid's private route lifecycle. Silent route death, the single-active-route constraint, and the undocumented self-import requirement each caused multi-day debugging sessions. The resulting workarounds (readiness barrier, reactive refresh, DHT-backed route exchange) are correct but fragile. These are framework maturity issues rather than fundamental limitations of the approach.

If starting again, the author would begin MPC integration earlier (Milestone 1 rather than Milestone 2) and spend less time on the UI. The headless binary and benchmark harness proved far more valuable for demonstrating and measuring the system than the desktop interface. The author would also invest more time in threat modelling and, given sufficient schedule, attempt stretch goals such as implementing malicious nodes to generate real adversarial test data.

References

Araki, T., A. Barak, J. Furukawa, T. Lichter, Y. Lindell, A. Nof, K. Ohara, A. Watzman and O. Weinstein (2017). 'Optimized Honest-Majority MPC for Malicious Adversaries — Breaking the 1 Billion-Gate Per Second Barrier'. In: *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 843–862. DOI: 10.1109/SP.2017.15.

Benet, J. (2014). *IPFS — Content Addressed, Versioned, P2P File System*. arXiv: 1407.3561 [cs.NI]. URL: https://arxiv.org/abs/1407.3561 (accessed on 02/04/2026).

Bogetoft, P., D. L. Christensen, I. Damgård, M. Geisler, T. Jakobsen, M. Krøigaard, J. D. Nielsen, J. B. Nielsen, K. Nielsen, J. Pagter, M. Schwartzbach and T. Toft (2009). 'Secure Multiparty Computation Goes Live'. In: *Financial Cryptography and Data Security*. Ed. by R. Dingledine and P. Golle. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 325–343. ISBN: 978-3-642-03549-4. DOI: 10.1007/978-3-642-03549-4_20. URL: https://link.springer.com/chapter/10.1007/978-3-642-03549-4_20.

Daian, P., S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach and A. Juels (2019). *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges*. DOI: 10.1109/SP40000.2020.00040. arXiv: 1904.05234 [cs.CR]. URL: https://arxiv.org/abs/1904.05234.

Damgård, I., V. Pastro, N. P. Smart and S. Zakarias (2011). *Multiparty Computation from Somewhat Homomorphic Encryption*. Cryptology ePrint Archive, Paper 2011/535. URL: https://eprint.iacr.org/2011/535.

Dingledine, R., N. Mathewson and P. Syverson (2004). 'Tor: The Second-Generation Onion Router'. In: *Proceedings of the 13th USENIX Security Symposium*. USENIX Association, pp. 303–320. URL: https://svn-archive.torproject.org/svn/projects/design-paper/tor-design.pdf (accessed on 02/04/2026).

eBay Help (2025). *How bidding works*. URL: https://www.ebay.co.uk/help/buying/bidding/bidding?id=4003 (accessed on 29/10/2025).

eBay Inc. (2025a). *Our Company*. URL: https://www.ebayinc.com/company/ (accessed on 27/03/2026).

– (2025b). *Our History*. URL: https://www.ebayinc.com/company/our-history/ (accessed on 27/03/2026).

Evans, D., V. Kolesnikov and M. Rosulek (December 2018). 'A Pragmatic Introduction to Secure Multi-Party Computation'. In: *Found. Trends Priv. Secur.* 2.2–3, pp. 70–246. ISSN: 2474-1558. DOI: 10.1561/3300000019. URL: https://doi.org/10.1561/3300000019.

Feng, Dengguo and Yang, Kang (2022). 'Concretely efficient secure multi-party computation protocols: survey and more'. In: *Security and Safety* 1, p. 2021001. DOI: 10.1051/sands/2021001. URL: https://doi.org/10.1051/sands/2021001.

Jackson, K. (2024). *Add Veilid Server Docker Container Implementation*. Draft merge request. URL: https://gitlab.com/veilid/veilid/-/merge_requests/444 (accessed on 08/04/2026).

Keller, M. (2020). 'MP-SPDZ: A Versatile Framework for Multi-Party Computation'. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. CCS '20. Virtual Event, USA: Association for Computing Machinery, pp. 1575–1590. ISBN: 9781450370899. DOI: 10.1145/3372297.3417872. URL: https://doi.org/10.1145/3372297.3417872.

Keller, M., E. Orsini and P. Scholl (2016). 'MASCOT: Faster Malicious Arithmetic Secure Computation with Oblivious Transfer'. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS '16. New York, NY, USA: Association for Computing Machinery, pp. 830–842. DOI: 10.1145/2976749.2978357.

Li, X., B. Zhao, G. Yang, T. Xiang, J. Weng and R. H. Deng (2023). *A Survey of Secure Computation Using Trusted Execution Environments*. DOI: 10.48550/arXiv.2302.12150. arXiv: 2302.12150 [cs.CR]. URL: https://arxiv.org/abs/2302.12150.

Marshall, C. (2024). *Stigmerge: Anonymous Decentralized File Sharing over Veilid*. URL: https://github.com/cmars/stigmerge (accessed on 14/02/2026).

Meta/Facebook Help Center (2025). *Marketplace Help Center: Using Marketplace to buy and sell*. URL: https://www.facebook.com/help/1713241952104830 (accessed on 29/10/2025).

OpenSea (2025a). *OpenSea: NFT Marketplace and Token Trading*. URL: https://opensea.io/ (accessed on 26/03/2026).

– (2025b). *Which blockchains are compatible with OpenSea?* URL: https://support.opensea.io/en/articles/8867082-which-blockchains-are-compatible-with-opensea (accessed on 26/03/2026).

Rioux, C. (2025). *DHT record patterns and 0.6.0 route autopublish*. Veilid Discord (#veilid-core channel), 07 October 2025. URL: https://discord.com/invite/5Qx3B9eedU (accessed on 02/04/2026).

– (2026a). *IP NAT punishment behaviour*. Veilid Discord (#veilid-core channel), 15 February 2026. URL: https://discord.com/invite/5Qx3B9eedU (accessed on 27/03/2026).

– (2026b). *Private route improvements planned for 0.6.0*. Veilid Discord (#veilid-core channel), 03 January 2026. URL: https://discord.com/invite/5Qx3B9eedU (accessed on 02/04/2026).

Sako, K. (2000). 'An Auction Protocol Which Hides Bids of Losers'. In: *Public Key Cryptography*. Ed. by H. Imai and Y. Zheng. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 422–432. ISBN: 978-3-540-46588-1. DOI: 10.1007/978-3-540-46588-1_28. URL: https://link.springer.com/chapter/10.1007/978-3-540-46588-1_28.

Shamir, A. (1979). 'How to Share a Secret'. In: *Communications of the ACM* 22.11, pp. 612–613. DOI: 10.1145/359168.359176.

U.S. General Services Administration (2025). *Federal Acquisition Regulation, Part 14 — Sealed Bidding*. URL: https://www.acquisition.gov/far/part-14 (accessed on 26/03/2026).

Veilid Project (2025a). *Bootstrap Setup*. URL:
https://gitlab.com/veilid/veilid/-/blob/main/BOOTSTRAP-SETUP.md (accessed on 27/03/2026).

– (2025b). *Veilid — A Privacy-First Application Framework*.
URL: https://veilid.com (accessed on 08/04/2026).

– (2025c). *Veilid Developer Book*.
URL: https://veilid.gitlab.io/developer-book/ (accessed on 31/10/2025).

Zantout, B. and R. Haraty (2011). *I2P Data Communication System*. Tech. rep., pp. 401–409.
URL: https://geti2p.net/en/docs/how/tech-intro (accessed on 02/04/2026).

Zhao, C., S. Zhao, M. Zhao, Z. Chen, C.-Z. Gao, H. Li and Y.-a. Tan (2019).
'Secure Multi-Party Computation: Theory, practice and applications'.
In: *Information Sciences* 476, pp. 357–372. ISSN: 0020-0255. DOI: 10.1016/j.ins.2018.10.024.
URL: https://www.sciencedirect.com/science/article/pii/S0020025518308338.

Appendix A Project Definition Document

Project Definition Document

November 10, 2025

Project Title: Decentralized Peer-to-Peer SMPC Sealed-Bid Auction App over the Veilid framework

Degree Programme: BSc (Hons) Computer Science

Project Consultant: Martin Nyx Brain

By: Anker Rasmussen

anker.rasmussen@city.ac.uk

Category: Academic Client Project

Subcategory: Application Development

Word count: 1631

Proprietary Interests: None

Contents

1 Proposal **3**

1.1 Problem to be Solved 3

1.2 Project Objectives 4

1.3 Project Beneficiaries 5

1.4 Project Plan 5

1.5 Risk Matrix (Probability–Severity–Mitigation) 8

1.6 Legal, Social, Ethical and Professional Considerations 9

1.7 References 10

1.8 Appendix: Research Ethics Form 11

1.9 Appendix: Client Description Sheet 14

1 Proposal

1.1 Problem to be Solved

Popular online consumer marketplaces either use *open, incrementally visible bidding* or *no auctions at all*, which undermines sealed-bid privacy and fairness. Current popular marketplaces such as eBay and Facebook Marketplace follow two distinct patterns. eBay's proxy-bidding model reveals live bid progression and awards the item to the highest bid at close [1]. Facebook Marketplace, by contrast, is a listing-plus-messaging workflow rather than a formal auction protocol, with weak buyer protections [2]. Public blockchain auctions add strong auditability but invite transaction ordering attacks (front-running/MEV) that can leak or distort bids during submission and reveal phases [3]. Cryptographic sealed-bid auctions can hide non-winning bids entirely. Deploying a protocol without a trusted auctioneer requires secure multiparty computation (MPC). MPC has been demonstrated in production (e.g., the Danish sugar-beet auction) and modern frameworks such as MP-SPDZ show practical performance across secret-sharing, HE, and garbled-circuit backends [4, 5, 6, 7, 8].

What is Veilid?

Veilid is a privacy-first, open-source, peer-to-peer application framework. Each app embeds a node into a global overlay where peers are equal (no privileged relays), connections are end-to-end encrypted, and private routing obscures network locations. After a brief bootstrap, apps communicate directly over transports such as UDP/TCP/QUIC/Web and exchange data via a secure DHT designed for mobile and desktop [9]. Veilid provides addressable, encrypted endpoints (public-key identities) and metadata-minimized communication, independent of any blockchain.

Why Veilid for this marketplace?

In line with the project brief to *build an application using the Veilid framework*, this marketplace needs private identities, censorship-resistant transport, and NAT-friendly reachability without a trusted coordinator. Veilid provides exactly these: (i) addressable, encrypted endpoints keyed by public keys rather than IPs, (ii) a secure DHT for publishing listings and discovering MPC parties, and (iii) obfuscated routing that limits metadata exposure during bid submission and MPC setup [9]. Because Veilid is blockchain-agnostic, settlement can occur off-overlay (e.g., Monero testnet) while *coordination* and *communication*

remain private and decentralized. [10].

Project Goal

Build a **peer-to-peer sealed-bid marketplace** that (1) preserves bidder privacy by default, (2) mitigates front-running risk during bid submission, and (3) operates *without a central auctioneer*. The prototype combines MPC for winner/price computation with *Veilid* for identity, routing, and availability [9]. Payments clear on *Monero testnet* after the MPC outcome is published. This demonstrates the full flow without handling real funds while inheriting the confidential-transaction model from the CryptoNote design (Monero precursor) [10].

1.2 Project Objectives

- **Main objective.** This project shall deliver a working peer-to-peer marketplace application on the Veilid network that exclusively supports sealed-bid listings and private content unlock for the winning bidder.
- **Testable sub-objectives.**
  - *Listings and purchases.*
    Implement end-to-end flows: create listing → submit bid → determine winner → complete purchase.
    Test: demo run and automated integration tests covering success/edge cases (invalid bid, tie, timeout).
  - *Sealed-bid via MPC.*
    Integrate a multi-party computation protocol to select the highest valid bid without revealing non-winning bids.
    Test: unit tests with mocked parties. Tests to show loser-bid privacy with reproducible benchmarks for N bidders.
  - *MPC-gated decryption.*
    Ideally: Bind winner selection to content access: the MPC that selects the highest valid bid also releases a winner-only decryption capability for the listing (K via 1-out-of-n OT(Oblivious Transfer)) only if settlement is confirmed on testnet.
    Test: unit tests showing that non-winners cannot recover K. Create integration test with mocked `paid`=0/1. Testnet demo where confirmation flips release.
  - *Settlement on Monero Testnet.*
    Simulate network init, fund wallets, and programmatic "real" transactions for deposits/escrow/release.

Test: scripted testnet transactions with confirmations & failure handling.

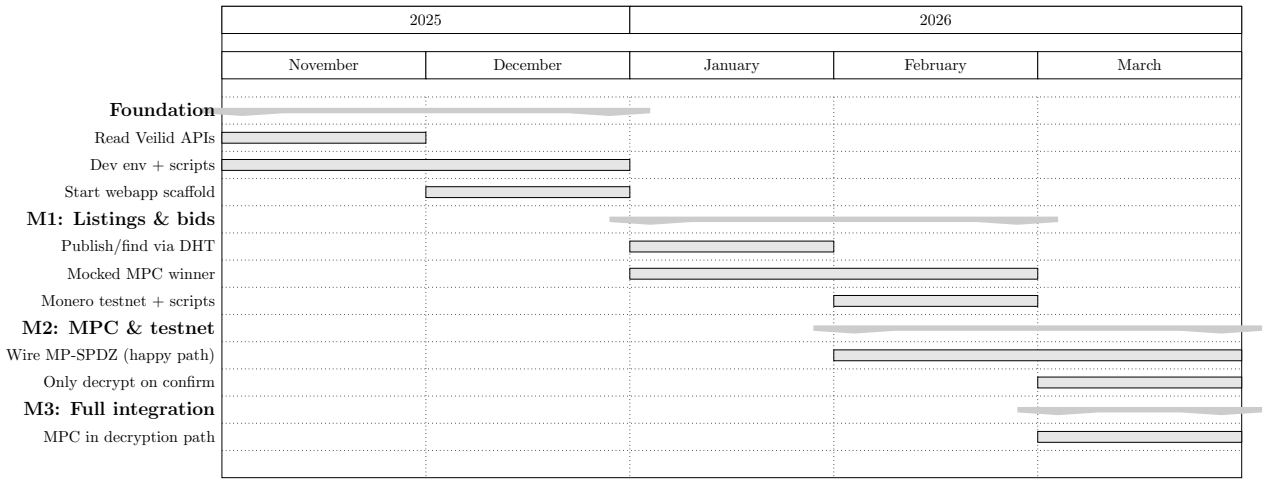
1.3 Project Beneficiaries

- **Project supervisor:** Gain a proof-of-concept application that utilizes Veilid's framework (keyed identities, secure DHT, obfuscated routing) with a non-toy use case. Deliverables (repo and demo video, alongside shell scripts) support future supervision of similar projects.
- **Veilid development team/Veilid Community:** A real world application that demonstrates capabilities of Veilid when integrated with other academic projects. Potential upstream bugfixes if warranted.
- **Marketplace users - research oriented (prototype).** Sellers get fairer price discovery without leaking losing bids. Bidders gain privacy and reduced front-running risk. In real-world deployments, at-risk users (e.g., in high-censorship environments) could benefit from metadata-minimising coordination.
- **End users in the future.** A metadata-minimising, sealed-bid coordination layer could be adapted for *safer* exchange of sensitive information (e.g., purchasing or rewarding data disclosures) where overt communication is filtered or surveilled.

1.4 Project Plan

- **Foundation (Nov-Dec 2025): Understand & set up Veilid. Digest current literature**
  - Read Veilid API spec & pull core code segments from *veilidchat*.
  - Create and ready a platform agnostic development environment with scripts to boot local Veilid nodes (to aid my cross-platform development - x86 Linux/MacOS ARM).
  - Begin development of webapp, light tests to see dev env works before implementing any of the features to be delivered. Stack: C++ to WebAssembly with a TypeScript UI.
- **Milestone 1 (mid Jan–Feb 2026): Listings & bids over Veilid**
  - Listing publish/discovery over secure DHT & private bid submission path.
  - Mocked MPC winner & basic content encryption at listing. Create happy-path E2E test.
  - Spool up private Monero testnet and understand integration with Veilid.
  - Update Monero testnet script & modify to fit usecase. Integrate with deployment script of dev network.

- **Milestone 2 (late Feb–Mar 2026): MPC & testnet**
  - Integrate real MPC (MP-SPDZ) and test happy path.
  - Fully integrate Monero testnet scripts. Implement Winner-only decrypt on confirmation.
  - E2E demo with simple implementation of MP-SPDZ + testnet.
- **Milestone 3 (late Mar 2026): Fully integrated MPC in Decryption**
  - (Attempt to) fully integrate MPC to the decryption algorithm.
  - Make test scripts/infrastructure spool up for project demo (dockerized nodes on one device).
- **Deliverables (Apr 2026): Package & submit**
  - Wrap up writing dissertation
  - Submit final report & demo video.


2025
2026
November
December
January
February
March
Foundation
Read Veilid APIs
Dev env + scripts
Start webapp scaffold
M1: Listings & bids
Publish/find via DHT
Mocked MPC winner
Monero testnet + scripts
M2: MPC & testnet
Wire MP-SPDZ (happy path)
Only decrypt on confirm
M3: Full integration
MPC in decryption path

1.5 Risk Matrix (Probability–Severity–Mitigation)

Legend: L = Low, M = Medium, H = High.

Risk	Prob.	Sev.	Mitigation
Veilid API or reusing code from `veilidchat` is harder than expected	M	H	Build a small wrapper with four actions: publish, find, send, receive. Start by running `veilidchat` as a separate helper and talk to it with simple text messages. If the direct API later works, replace the helper. Write clear build steps and ensure code is commented and documented to reduce code entropy (loss of context).
Understanding MPC and integrating MP–SPDZ takes longer than planned	M	H	Follow a short learning plan: run the MP–SPDZ tutorial and a simple max-of-N example. Treat MPC as a small command-line tool with JSON in/out so the app can call it easily. Run all parties locally first, before implementing dockerised build scripts.
Integrating Veilid–MPC–Monero (HTTP/JSON)	M	H	Standardise on HTTP+JSON: use Monero's native JSON-RPC, expose MPC as a localhost service with a minimal OpenAPI/JSON-Schema-defined surface. Generate clients, add contract tests before E2E, enforce timeouts/retries and idempotent ops. Keep MPC/Monero as separate processes.
Containerisation & multi-arch builds (Docker/Compose)	M	H	Build small images from fixed base versions. Use plain HTTP/JSON between services so they connect cleanly. Start everything with a single Compose file (Veilid app, MPC sidecar, Monero sidecar). Keep chain data in named volumes. To reduce surprises, target one platform for the demo (64-bit Linux) and have CI build and run the images throughout development. Add health checks and restart policies, and run as a non-root user with secrets loaded from a .env file.

Continued on next page

Risk	Prob.	Sev.	Mitigation
P2P connectivity on eduroam	M	M	Dockerise apps on same-LAN with docker multi-node compose & keep a pre-recorded run with logs & video for assessment.
Toolchain/portability (macOS vs Linux, dependencies)	M	M	Containerised builds with fixed minimal versions. CI on x86_64 Linux on GitHub Actions, avoiding platform-specific flags. reproducible build script for different architectures.
Hardware failure / data loss	L	H	Keep dissertation sourcefiles in `.tex` within Git, push & pull frequently. Keep remote backups for code in GitHub & GitLab. Maximum loss will be minimal

1.6 Legal, Social, Ethical and Professional Considerations

Project category (per handbook 4.5.7): Category 1 prototype not intended for distribution. Operation is confined to a local Veilid dev overlay and Monero testnet. No mainnet, no PII, and no real funds.

Scope. Proof-of-concept only: runs on testnet for Monero, but runs on a localized Veilid dev environment.

Statement on illicit uses. I explicitly condemn any attempt to adapt this research for illicit trade, exploitation, or other illegal activity. The work is motivated by technical research questions (privacy-preserving auction protocols, MPC integration, and overlay networking) rather than any intent to facilitate wrongdoing. All design, testing and evaluation are performed within controlled, local test environments to prevent misuse.

Principal risk (dual-use). A private marketplace design could in principle be repurposed to facilitate illicit exchange if deployed without appropriate safeguards.

Mitigations to be implemented in this prototype.

- No mainnet support. Script will have localized test peers (on same LAN) with public bootstrap for Veilid disabled (dev veilid network).
- No custody of real funds, testnet. Payment "confirmations" exist only on Monero's testnet, and wallets/the network are created/destroyed with the run script.

Legal (licensing). Project is released under **MPL-2.0** (file-level copyleft). Veilid

already exists under **MPL-2.0**. any modified Veilid files are published under MPL as required. No copyleft extends to independent project files beyond their chosen license.

Professional conduct. No human participants.

Residual risk and fallback. Code could be forked and modified. Reduce this by omitting deployment scripts on live Veilid + live Monero. If further legal/ethical concerns arise, disable network capabilities and run on a local area network.

1.7 References

References

[1] eBay Help, *How bidding works*, Accessed 29 Oct 2025, 2025. [Online]. Available: https://www.ebay.co.uk/help/buying/bidding/bidding?id=4003.

[2] Meta/Facebook Help Center, *Marketplace help center: Using marketplace to buy and sell*, Accessed 29 Oct 2025, 2025. [Online]. Available: https://www.facebook.com/help/1713241952104830.

[3] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, *Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges*, 2019. DOI: 10.1109/SP40000.2020.00040. arXiv: 1904.05234 [cs.CR]. [Online]. Available: https://arxiv.org/abs/1904.05234.

[4] P. Bogetoft, D. L. Christensen, I. Damgård, M. Geisler, T. Jakobsen, M. Krøigaard, J. D. Nielsen, J. B. Nielsen, K. Nielsen, J. Pagter, M. Schwartzbach, and T. Toft, "Secure multiparty computation goes live," in *Financial Cryptography and Data Security*, R. Dingledine and P. Golle, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 325–343, ISBN: 978-3-642-03549-4. DOI: 10.1007/978-3-642-03549-4_20. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-03549-4_20.

[5] M. Keller, "Mp-spdz: A versatile framework for multi-party computation," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '20, Virtual Event, USA: Association for Computing Machinery, 2020, pp. 1575–1590, ISBN: 9781450370899. DOI: 10.1145/3372297.3417872. [Online]. Available: https://doi.org/10.1145/3372297.3417872.

[6] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Advances in Cryptology – CRYPTO 2012*, R. Safavi-Naini and R. Canetti, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 643–662, ISBN: 978-3-642-32009-5.

[7] D. Evans, V. Kolesnikov, and M. Rosulek, "A pragmatic introduction to secure multi-party computation," *Found. Trends Priv. Secur.*, vol. 2, no. 2–3, pp. 70–246, Dec. 2018, ISSN: 2474-1558. DOI: 10.1561/3300000019. [Online]. Available: https://doi.org/10.1561/3300000019.

[8] K. Sako, "An auction protocol which hides bids of losers," in *Public Key Cryptography*, H. Imai and Y. Zheng, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 422–432, ISBN: 978-3-540-46588-1. DOI: 10.1007/978-3-540-46588-1_28. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-46588-1_28.

[9] Veilid Project. "Veilid developer book." Official developer documentation for the Veilid framework, Accessed: Oct. 31, 2025. [Online]. Available: https://veilid.gitlab.io/developer-book/.

[10] N. van Saberhagen, "Cryptonote v 2.0," Original CryptoNote whitepaper; commonly cited in Monero literature, Oct. 17, 2013. Accessed: Oct. 31, 2025. [Online]. Available: https://decred.org/research/saberhagen2013.pdf.

1.8 Appendix: Research Ethics Form

Research Ethics Review Form for BSc and MSci Projects

Computer Science Research Ethics Committee (CSREC)

http://www.city.ac.uk/department-computer-science/research-ethics

Undergraduate students undertaking their final project in the Department of Computer Science must consider the ethics of their project work and ensure that it complies with research ethics guidelines and the law for data protection. In some cases, a project will need approval from an ethics committee before it can proceed. Usually, but not always, this will be because the student is involving other people ("participants") in the project.

To ensure that they give appropriate consideration to ethical issues, all students must complete this form and attach it to their project definition document (PDD). There are two parts:

PART A: Ethics Checklist. All students must complete this part.
The checklist identifies whether the project requires ethical approval and, if so, where to apply for approval.

PART B: Ethics Proportionate Review Form. Students who have answered "no" to all questions in A1, A2 and A3 and "yes" to question 4 in A4 in the ethics checklist must complete part B as well. The project supervisor or consultant has delegated authority to provide approval in such cases that are considered to involve MINIMAL risk. The approval may be ***provisional*** – *identifying the planned work with human end user participants as* likely to involve MINIMAL RISK. In such cases you must additionally seek ***full approval*** from the supervisor or consultant as the project progresses and details are established. You must obtain ***full approval*** in writing, before recruiting and engaging with human end users participants for your project.

A.1 If you answer YES to any of the questions in this block, your consultant/supervisor must have obtained approval for the project from an appropriate external ethics committee, and you need to have received written confirmation of this from him/her. Students cannot themselves apply for ethics approval in this case as the project is considered high risk". This type of research is not covered by City's process, and external approval from an appropriate institution is required.		*Delete as appropriate*
1.1	Does your research require approval from the National Research Ethics Service (NRES)?	**NO**
1.2	Will you recruit participants who are covered by the Mental Capacity Act 2005?	**NO**
1.3	Will you recruit any participants who are covered by the Criminal Justice System, for example, people on remand, prisoners and those on probation?	**NO**
A.2 If you answer YES to any of the questions in this block your consultant/supervisor must have obtained appropriate ethics committee approval		*Delete as appropriate*
2.1	Does your research involve participants who are unable to give informed consent? *For example, people who may have a degree of learning disability or mental health problem, that means they are unable to make an informed decision on their own behalf.*	**NO**
2.2	Is there a risk that your research might lead to disclosures from participants concerning their involvement in illegal activities?	**NO**
2.3	Is there a risk that obscene and or illegal material may need to be accessed for your research study (including online content and other material)?	**NO**
2.4	Does your project involve participants disclosing information about protected characteristics (as identified by the Equality Act 2010)?	**NO**

	For example: racial or ethnic origin; political opinions; religious beliefs; trade union membership; physical or mental health; sexual life; criminal offences and proceedings	
2.5	Does your research involve you travelling to another country outside of the UK, where the Foreign & Commonwealth Office has issued a travel warning that affects the area in which you will study? *Please check the latest guidance from the FCO - http://www.fco.gov.uk/en/*	**NO**
2.6	Does your research involve invasive or intrusive procedures? *These may include, but are not limited to, electrical stimulation, heat, cold or bruising.*	**NO**
2.7	Does your research involve animals?	**NO**
2.8	Does your research involve the administration of drugs, placebos or other substances to study participants?	**NO**
A.3 If you answer YES to any of the questions in this block, then unless you are applying to an external ethics committee or the Senate Research Ethics Committee (SREC), you must apply for approval from the Computer Science Research Ethics Committee (CSREC) through Research Ethics Online - https://researchmanager.city.ac.uk/. Depending on the level of risk associated with your application, it may be referred to the Senate Research Ethics Committee (SREC).		*Delete as appropriate*
3.1	Does your research involve participants who are under the age of 18?	**NO**
3.2	Does your research involve adults who are vulnerable because of their social, psychological or medical circumstances (vulnerable adults)? *This includes adults with cognitive and / or learning disabilities, adults with physical disabilities and older people.*	**NO**
3.3	Are participants recruited because they are staff or students of City, University of London? *For example, students studying on a particular course or module.* *If yes, then approval is also required from the Head of Department or Programme Director.*	**NO**
3.4	Does your research involve intentional deception of participants?	**NO**
3.5	Does your research involve participants taking part without their informed consent?	**NO**
3.5	Is the risk posed to participants greater than that in normal working life?	**NO**
3.7	Is the risk posed to you, the researcher(s), greater than that in normal working life?	**NO**
A.4 If you answer YES to the following question and your answers to all other questions in sections A1, A2 and A3 are NO, then your project is deemed to be of MINIMAL RISK. **If this is the case, then you can apply for approval through your supervisor under PROPORTIONATE REVIEW. You do so by completing PART B of this form.** **If you have answered NO to all questions on this form, then your project does not require ethical approval. You should submit and retain this form as evidence of this.**		*Delete as appropriate*
4	Does your project involve human participants or their identifiable personal data? *For example, as interviewees, respondents to a survey or participants in testing.*	**NO**

1.9 Appendix: Client Description Sheet

“‘ This project involves working with an academic at City St George’s in the department of computer science. The academic will supervise and guide your work alongside being your consultant and marker.

Contact details:

Staff name:	Martin Nyx Brain
Email address:	martin.brain@citystgeorges.ac.uk

Project title: Application Development Using the Veilid Application Framework

Project description:	Keywords: networking, privacy, anonymity, peer-to-peer, distributed applications, anti-surveillance, cryptography
	Description: The Veilid framework (https://veilid.com/) is a recent developed collection of protocols to build distributed, private applications. It builds on the onion routing ideas used by Tor and I2P as well as distributed hash tables and routing as used in IPFS and others. It aims to make developing highly-scalable, distributed, private applications easy. Veilid is still very much under active development, so this project would aim to create a proof-of-concept / example application to demonstrate some of its features. Possible applications:
	• A multi-user notepad / text editor.
	• An anonymous voting system.
	• An anonymous question submission system.
	• A SOCKS proxy
	Plan:
	1. Learn the key concepts involved including onion routing and distributed hash tables.
	`2. Set up a Veilid node and try out the demo application.`
	
	`3. Develop your own application!`
	Skills: Knowledge of networking and cryptography, software development on Linux, reasonable programming ability.

Ideal Project Outcomes:	1. A network application that uses the Veilid Framework. 2. A demonstration of the application.
Additional Information	Contact Martin for more information.

Appendix B Reuse Summary

This appendix documents all third-party software reused in this project, as required by the handbook.

Frameworks and Libraries

Component	Version	Licence	Usage
veilid-core	(local build)	MPL-2.0	P2P networking, DHT, private routing
MP-SPDZ	(local build)	BSD-3-Clause	MASCOT MPC protocol execution
Dioxus	0.7.2	MIT/Apache-2.0	Desktop UI framework
tokio	1.x	MIT	Async runtime
serde	1.0	MIT/Apache-2.0	Serialization framework
ciborium	0.2	MIT/Apache-2.0	CBOR serialization for DHT records
bincode	1.3	MIT	Binary serialization for messages
sha2	0.10	MIT/Apache-2.0	SHA-256 hashing (bid commitments)
aes-gcm	0.10	MIT/Apache-2.0	AES-256-GCM content encryption
rand	0.8	MIT/Apache-2.0	Random number generation
anyhow	1.0	MIT/Apache-2.0	Error handling
thiserror	1.0	MIT/Apache-2.0	Error type derivation
tracing	0.1	MIT	Structured logging
async-trait	0.1	MIT/Apache-2.0	Async trait support

Table B.1: Third-party libraries used in the Lattice crate.

Development Tools

- **Docker devnet**: Initial devnet environment adapted from a Veilid community merge request (Jackson 2024); later replaced by the Veilid Playground (process-based, no Docker).
- **cargo-nextest**: Test runner for parallel and sequential test execution.
- **libipspoof.so**: LD_PRELOAD library from Veilid devcontainer for IP address spoofing in devnet. Source: Veilid repository (MPL-2.0).

Code Reuse from Reference Applications

The Veilid project's `veilidchat` application was studied as a reference for Veilid API usage patterns (DHT record creation, routing context setup, message handling). No code was directly copied; API patterns were adapted for the auction use case.

Stigmerge (Marshall 2024), a Veilid-based file-sharing application, was studied for its use of `app_call` over private routes for reliable message delivery. The delivery pattern was adapted for the MPC tunnel proxy; Stigmerge's DHT-write approach was not used.

Appendix C Supervisory Meeting Minutes

Meeting 1 — [29/01/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: How to handle decryption and key delivery between auction participants. Communication patterns over Veilid nodes: `app_broadcast` vs. `app_message` for different message types. Initial discussion of what the MPC program needs to compute and how results are communicated back to participants.

Actions: Investigate Veilid messaging primitives and determine which is appropriate for each communication pattern. Sketch the MPC program requirements.

Meeting 2 — [05/02/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: Secret transfer between parties is leaky: transferring the decryption key reveals information about who won. Broader metadata leakage concerns: when should secrets be shared with which parties, and how to obscure participant identities throughout the protocol.

Actions: Research metadata leakage vectors. Design the secret transfer step to minimise information revealed to non-winning parties.

Meeting 3 — [12/02/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: Security model for auction results: only the winner and seller should receive sensitive information (decryption key and bid values). All other metadata exposure should be minimised.

Actions: Design the post-MPC verification flow around this principle: seller learns winner identity and winning bid; all other parties learn only whether they won or lost.

Meeting 4 — [19/02/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: Arrived with semi-honest (Shamir) as the default protocol choice assuming honest majority. Discussed whether to change the MPC auction algorithm, and what exactly needs to be computed inside the MPC program. Key question: how to handle bid commitment and how to prove the winner actually bid what they claim.

Actions: Define the commitment scheme (hash-based) and design a post-MPC verification challenge so the seller can confirm the winning bid matches the commitment.

Meeting 5 — [26/02/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: DHT registry permissions: if all parties hold a modify key, any party can delete other listings. Proposed a registry-of-registries pattern, but open question on how to bootstrap discovery (rebroadcast?). Route reuse across auctions: if a participant joins two auctions on the same route, this could enable deanonymisation. MASCOT vs. Shamir selection: MP-SPDZ is somewhat opaque regarding internal behaviour. Party count and devnet sizing. Initial hypothesis that MASCOT was network-bound on the Veilid transport layer (later found to be incorrect: the bottleneck was running three Tokio runtimes in a single process).

Actions: Investigate per-listing DHT records to avoid shared-registry permission conflicts. Ensure fresh routes per auction. Separate market nodes into individual OS processes.

Meeting 6 — [12/03/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: Martin asked to formalise the security model: protecting what, from whom, how, and under what assumptions. Party assignment: how to deterministically assign party IDs (0, 1, 2, ...) given that party 0 (seller) is privileged. Options discussed: sort by bid timestamp, sort by node ID. Must be deterministic so all parties independently agree on the same assignment.

Actions: Write up the threat model. Implement party assignment by bid timestamp (seller's auto-bid is earliest, guaranteeing party 0).

Meeting 7 — [19/03/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: Switched default protocol from Shamir to MASCOT, and from unsafe to private routing. Veilid v0.6.0 reportedly bringing significant routing changes. E2E test flakiness. Compare-and-swap is not a Veilid-native primitive and can only be approximated at the application layer; Martin suggested using a CRDT instead. SSL is only required with Shamir, not MASCOT (which uses OT-based key setup); system is fully reliant on Veilid's privacy layer, so if Veilid is compromised the MPC is also compromised. Concurrent writes to the same DHT subkey caused race conditions (no concurrent primitives in Veilid). Veilid does not support a multi-owner model for DHT records. The 32 KB DHT record limit means a single record can realistically support at most 256 bidders.

Actions: Implement CRDT-based (G-Set) bid registry. Accept Veilid as the sole trust boundary for transport encryption. Design around the single-owner DHT constraint.

Meeting 8 — [26/03/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: CRDT implementation: for true CRDTs each node needs to maintain its own copy, with state stored as subkeys on its own DHT record and a merge layer baked into the application layer. Current approach was hacky with the seller centralising state. Martin's feedback: local copies should converge via merge, not via a central authority. Veilid's DHT still does not support multi-writer records cleanly.

Actions: Refactor bid registry to use G-Set merge semantics with per-node owned records. Remove seller as central state authority.

Meeting 9 — [02/04/2026]

Attendees: Anker Rasmussen, Martin Brain

Discussion: Dissertation writing strategy: start writing now, begin with the conclusion and work backwards. Implementation priority: finish the CRDT refactor first, then write up results.

Actions: Implement G-Set CRDT merge. Begin drafting conclusion and results chapters.

Appendix D Generative AI Usage

This appendix declares the use of generative AI tools during the project, in accordance with the department's policy on generative AI (Handbook Section 9.7).

Tools Used

Tool	Provider
Claude Code (Opus 4)	Anthropic

Table D.1: Generative AI tools used.

How AI Was Used

The following describes how Claude Code was used during the project, in chronological order of development. In each case the prompt was given within the context of the full project codebase. All core application logic (auction coordination, MPC orchestration, Veilid integration, cryptographic protocols) was designed and written by the author. Claude Code was used primarily for infrastructure and tooling (devnet setup, build scripts, UI), refactoring, and debugging. All code, whether AI-generated or hand-written, was reviewed, tested, and committed by the author. All prose in this report was written by the author; Claude Code was used only for cross-referencing and gap analysis.

1. **Docker devnet bootstrap.**
   Prompt: "The Veilid devnet merge request doesn't work. Nodes fail to bootstrap. Here's the docker-compose and the error output: [pasted logs]."
   Output: Claude iterated on the Docker Compose configuration, bootstrap entrypoint scripts, network key settings, and DNS configuration.
   What was done: The author tested each iteration against the actual Veilid bootstrap process, feeding error logs back across multiple rounds until the 20-node devnet started reliably.

2. **IP spoofing LD_PRELOAD library.**
   Prompt: "Write a C LD_PRELOAD library that intercepts `bind()`, `connect()`, `sendto()`, and `recvfrom()` to translate fake public IPs (1.2.3.x) to localhost, so multiple Veilid nodes can run on one machine."
   Output: Claude generated `ip_spoof.c` with the socket interception logic and address translation table.
   What was done: The author reviewed the implementation, tested it against the devnet, and later extended it with NAT table support for Tailscale. The library was subsequently ported to Rust by Claude for the veilid-playground project.

3. **Connecting market nodes to the devnet.**
   Prompt: "Now adapt the market application's Veilid node config to connect to this dockerised devnet."
   Output: Claude generated the node configuration changes: network key, bootstrap address, port offsets, and capability flags.
   What was done: The author tested the connection against the running devnet and iterated on capability settings.

4. **Dioxus UI: initial layout.**
   Prompt: "Build a Dioxus desktop UI with three panels: marketplace listings, active auctions, and a bid submission form. Use the AppState struct in src/app/state.rs."
   Output: Claude generated the Dioxus component hierarchy (`src/app/components.rs`, `src/app/mod.rs`) with state bindings.
   What was done: The author reviewed the generated code, adjusted the layout, and integrated it with the Veilid node lifecycle.

5. **Dioxus UI: stylesheet refactor.**
   Prompt: "Refactor the inline styles into a Dioxus stylesheet."
   Output: Claude extracted all inline styles into a centralised stylesheet.
   What was done: The author reviewed and committed the refactor.

6. **Build scripting: 3-node demo.**
   Prompt: "Write a Makefile target that starts a 3-node demo: builds the release binary, starts the devnet, then launches 3 market nodes with offsets 20–22 using LD_PRELOAD."
   Output: Claude generated the `make demo` target with process management and SIGTERM cleanup.
   What was done: The author tested the target, fixed a platform-specific issue on Linux, and committed it.

7. **Unit and integration test harness.**
   Prompt: "Set up a mock-based test infrastructure so I can simulate N-party auctions without a real devnet."
   Output: Claude generated `MockDht`, `MockTime`, `MockRandom`, and the `MultiPartyHarness` for running full auction flows in under 1 s.
   What was done: The author reviewed the mock implementations, verified they matched real Veilid DHT semantics, and built integration tests on top of them.

8. **Two-tier registry refactor.**
   Prompt: "The shared DHT registry lets any party delete others' listings. Refactor to a two-tier registry-of-registries pattern with per-seller owned records."
   Output: Claude restructured the registry into per-seller DHT records with a top-level catalog.
   What was done: The author verified the permission model and tested multi-seller scenarios.

9. **Codebase cleanup and refactoring.**
   Prompt: "Remove all dead code, collapse unnecessary generic parameters, deduplicate test helpers, and get clippy passing with pedantic lints."
   Output: Claude produced a series of refactoring patches: collapsing `AuctionLogic<D,T,M,C>` to `<D,C>`, deleting unused traits and mock self-tests, renaming `MpcSidecar` to `MpcTunnelProxy`, deduplicating broadcast helpers, unifying `MockDht` types, and dropping `async-std` in favour of Tokio. Net result: $-2{,}877$ lines across six commits.
   What was done: The author reviewed each patch, verified all 169 unit and 63 integration tests still passed, and committed as six logical commits.

10. **Error hardening.**
    Prompt: "Audit the codebase for panics and unwraps in production code paths. Replace with proper error handling."
    Output: Claude identified unsafe `unwrap()` calls in node configuration, string indexing, and insecure storage setup, and generated replacements using `Result` propagation.
    What was done: The author reviewed each change, confirmed no behaviour regressions, and committed the fixes.

11. **Strict clippy lints.**
    Prompt: "Enable pedantic, nursery, and cargo clippy lint groups and fix everything."
    Output: Claude enabled the lint groups and fixed the resulting warnings across the codebase.
    What was done: The author reviewed the changes and committed them.

12. **E2E test debugging: sequential auctions.**
    Prompt: "All 3 E2E tests are failing with route timeout on the second auction. The first auction completes fine. Here's the log output: [pasted test log]."
    Output: Claude identified that broadcast routes from the first auction were stale by the time the second auction started, and suggested adding a route refresh after MPC completion.
    What was done: The author implemented the `needs_route_refresh` AtomicBool flag in `MpcOrchestrator` based on this diagnosis, then verified the fix resolved the failure.

13. **E2E test debugging: concurrent auction deadlock.**
    Prompt: "Two auctions expiring at the same time cause a deadlock. Here's the log: [pasted log]."
    Output: Claude identified that nodes processed expired listings in different orders, and suggested sorting them deterministically by key before sequential processing.
    What was done: The author implemented the deterministic sort and verified the deadlock was resolved.

14. **Performance optimisation: MPC tunnel pipelining.**
    Prompt: "E2E tests take 900s. Profile the MPC tunnel and find the bottleneck."
    Output: Claude suggested pipelining `app_call` sends with a bounded semaphore (depth 8) instead of sequential send-wait cycles, and reducing polling intervals.
    What was done: The author implemented the pipeline in `MpcTunnelProxy`, tuned the semaphore depth experimentally, and measured a reduction from $\sim$900 s to $\sim$570 s.

15. **Dioxus UI: auction progress indicators.**
    Prompt: "Add visual feedback for the MPC phases: route collection, readiness barrier, and MPC execution."
    Output: Claude generated status indicators showing each phase's progress in the auction view.
    What was done: The author wired the indicators to the orchestrator state callbacks and tested them.

16. **Dioxus UI: live hex traffic panel.**
    Prompt: "Add a live hexdump display showing the encrypted MPC traffic flowing through the tunnel. Make it look nice."
    Output: Claude generated a formatted hex viewer component with colour-coded bytes and ASCII sidebar.
    What was done: The author integrated it into the auction detail view.

17. **Dioxus UI: dark theme and 3D lattice header.**
    Prompt: "Restyle the UI with a dark theme and add a rotating 3D lattice animation for the header."
    Output: Claude generated the dark-themed CSS and a WebGL lattice animation.
    What was done: The author reviewed and committed the styling changes.

18. **Dioxus UI: graceful shutdown.**
    Prompt: "Handle SIGTERM so the UI and Veilid node shut down cleanly instead of hanging."
    Output: Claude implemented signal handling with Tokio's signal listener and a clean shutdown sequence.
    What was done: The author tested the shutdown behaviour with `make demo` and committed it.

19. **Benchmark harness.**
    Prompt: "Write a benchmark binary that runs automated multi-party auction trials across configurable party counts and devnet sizes, with CSV output and resume support."
    Output: Claude generated the `bench-auction` binary and associated shell scripts.
    What was done: The author tested the harness across 40, 60, and 80-node devnets and committed it.

20. **Veilid playground migration.**
    Prompt: "Migrate the E2E test infrastructure from Docker to the veilid-playground (native processes with LD_PRELOAD)."
    Output: Claude adapted the test helpers to detect and use the playground instead of Docker.
    What was done: The author verified E2E tests passed on both Docker and playground backends.

21. **Security audit fixes.**
    Prompt: "Audit the codebase for security issues: race conditions, information leakage, serde misconfigurations."
    Output: Claude identified TOCTOU races in MPC session dedup and auction activation, serde annotation issues that could leak key/nonce material, and timing-leak vectors.
    What was done: The author reviewed each finding, committed the fixes, and verified tests still passed.

22. **Benchmark data processing and graphing.**
    Prompt: "Write Python scripts to parse the benchmark CSV files, compute medians and success rates, and output .dat files for pgfplots."
    Output: Claude generated the data processing scripts that produced the `.dat` files used by the pgfplots charts in Chapter 5.
    What was done: The author verified the computed values against the raw CSV data and adjusted chart formatting.

23. **LaTeX formatting.**
    Prompt: "Set up the dissertation LaTeX build: XeLaTeX with Aptos font, Harvard-style biblatex, word count automation, pgfplots for benchmark charts."
    Output: Claude configured the document preamble, bibliography style, and pgfplots chart templates.
    What was done: The author adjusted formatting to match handbook requirements and populated all content.

24. **Report gap analysis.**
    Prompt: “Compare my Ch5 Results against the codebase and benchmark CSVs. What’s missing or inconsistent?”
    Output: Claude flagged that several table values used approximate markers where exact data existed in the benchmark CSV, and that the warm/cold devnet distinction had no backing data.
    What was done: The author replaced approximate values with exact figures from the CSV and removed the unsupported warm/cold comparison.

Responsibility

The author takes full responsibility for the content, accuracy, and originality of this report and all project outputs. AI tools were used as assistants, not as authors. All claims, analyses, and conclusions are the author’s own.

Appendix E Veilid Developer Correspondence

This appendix reproduces excerpts from the public Veilid Discord server. These messages from Christien Rioux (DilDog), lead developer of the Veilid framework, informed architectural decisions in this project.

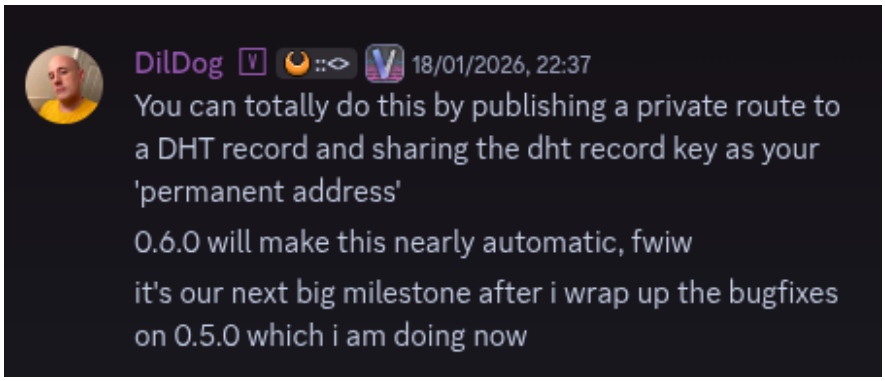


Figure E.1: DilDog on publishing private routes via DHT records as a “permanent address” pattern, and the planned v0.6.0 automation of this mechanism (January 2026).

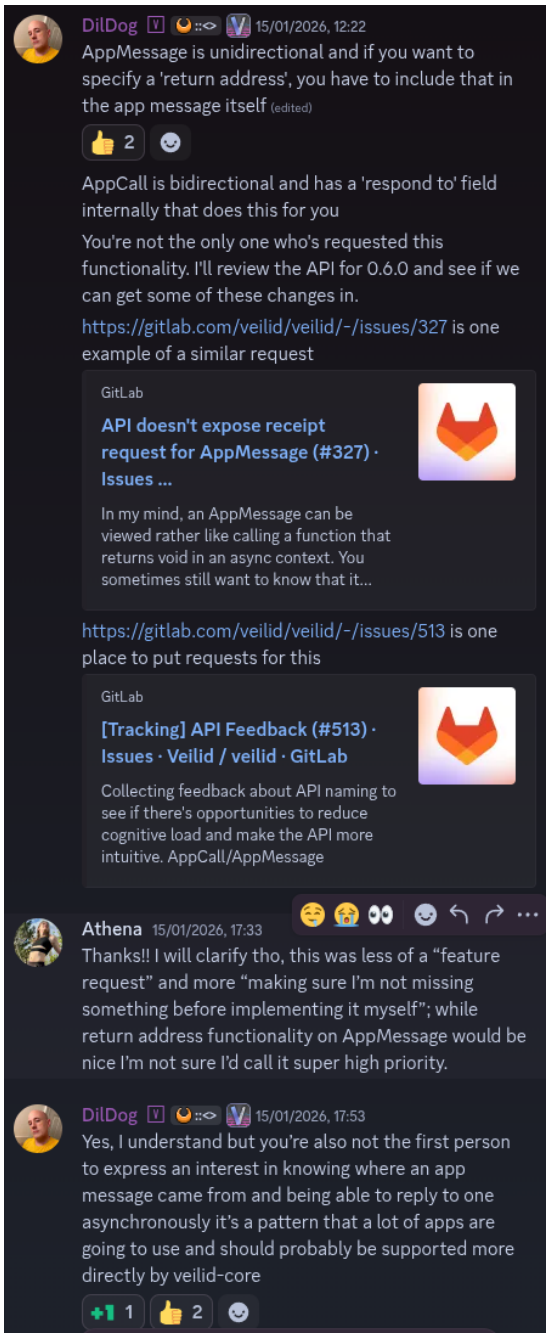


Figure E.2: Discussion of `app_call` vs. `app_message` semantics: `app_call` is bidirectional with a response, while `app_message` is fire-and-forget with no delivery guarantee.

DilDog 03/01/2026, 14:00

And yes, appcalls need to be returned quickly, there is an RPC timeout. We could probably parameterize that to allow for an arbitrary timeout though in this case. AppCall, unlike the more internal veilid rpc calls, should be more application-defined.

One way to deal with this right now is to include a response private route blob in an AppMessage.

which you then import and send another app message to

You mentioned including a DHT record as a place to send a response, and using watchvalue, which is totally fine as well. Just pointing out that as of 0.6.0, the 'route id' mechanism will actually be doing this in the background (kinda).

We'll be updating all of this and I'm taking people's feedback on that whole interface to improve on it for 0.6.0

in short:

- private routes need to be easier to use and require less boilerplate
- app calls and messages need more flexibility, app calls should be more intuitive, and doing things like 'replying' to app messages despite their unidirectionality should be easier, to simplify long-term rpc operations that have no timeout.
- DHT backing for private routes to keep them from dying, and to provide redundancy and reliability guarantees not present today.

(edited)

1

Figure E.3: DilDog on planned improvements to the Veilid API: private route resilience, provider routing, and DHT publishing of route blobs.

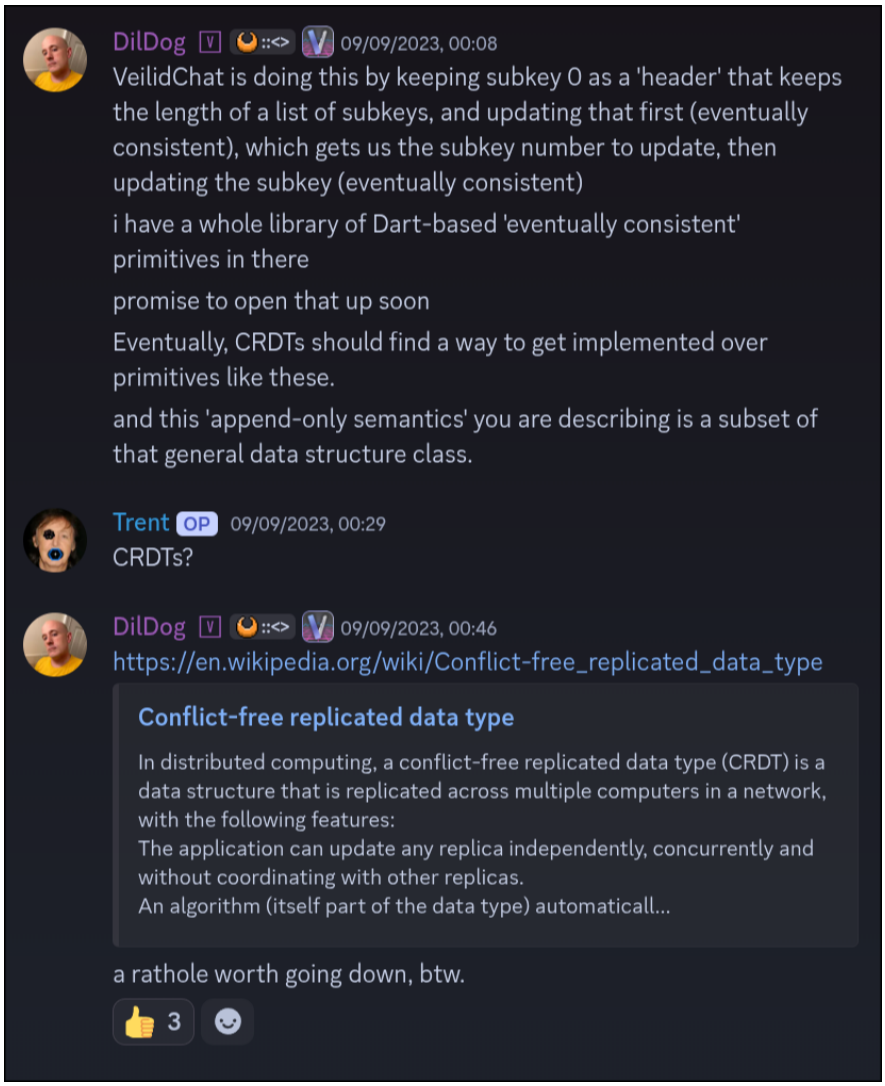


Figure E.4: DilDog on DHT record design patterns: using subkey 0 as a header for eventually consistent updates, and the relationship to CRDTs.

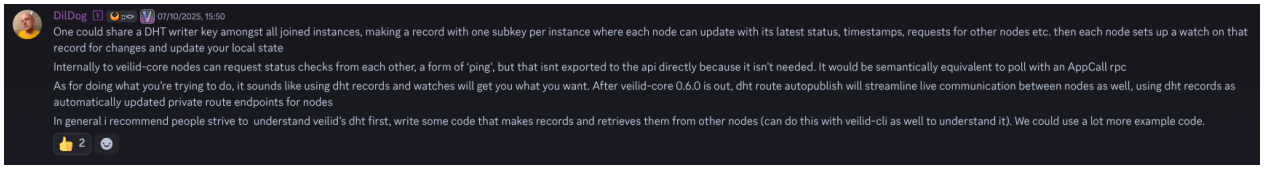


Figure E.5: DilDog on DHT record ownership and multi-writer patterns: each node can own a record and update its subkeys independently, with eventual consistency across the network.